



Multilevel Monte Carlo Methods and Their Applications in Options Pricing

Anton Waldron
BSc in Financial Mathematics
University of Limerick

Supervisor
Dr. Alan Hegarty
Department of Mathematics and Statistics
University of Limerick

Final Year Project
April 3, 2026

Abstract

In this project we examine the Multilevel Monte Carlo (MLMC) method and its applications in options pricing. First we introduce the main MLMC theorem and algorithm, and explain these algorithms and MLMC functions by comparing standard Monte Carlo and MLMC for estimating π . Following this we introduce the reader to a variety of different financial options and discuss some of their pricing attempts. We show that in practice MLMC can be 1000x cheaper than standard Monte Carlo. With regard to specific option types, we show that MLMC is applicable to Fixed Lookback options, based on the maximum of the stock price over a certain time interval. We use the Milstein method alongside a Brownian Bridge interpolant to more accurately capture the maximum of the stock price over the path, resulting in MLMC being approximately 136x cheaper than standard Monte Carlo for RMSE targets of 0.005, 0.01, 0.02 and 0.05. Finally we discuss American options and show that the LSM (Least Squares Monte Carlo) method can be used in conjunction with MLMC for an estimator that is 1.5x-2.5x cheaper than standard Monte Carlo.

Keywords: *Monte Carlo Methods, Multilevel Monte Carlo, Options Pricing, American Options, Lookback Options, Fixed Lookback Options, Brownian Motion, Brownian Bridging, LSM, Computational Finance*

Contents

Abstract	i
Table of contents	ii
List of figures	iv
1 Introduction	1
1.1 Introduction to Multilevel Monte Carlo	1
1.1.1 MLMC Theory	3
1.1.2 MLMC Algorithm and function	3
2 From theory to application	5
2.1 Convergence tests and plots explained	12
3 Options Pricing	13
3.1 What are options?	13
3.2 Options and their pricing	14
3.3 Brownian Motion	16
3.3.1 Brownian Bridging	17
3.4 Vanilla Options	19
3.4.1 Monte Carlo Simulation	19
3.4.2 A Multilevel Monte Carlo Implementation	20
3.5 Empirical Results on a Real Life Options Dataset	23
3.5.1 Empirical Results	23
3.5.2 <code>call_1(1, N, r, sig, K, S0)</code>	23
3.6 Lookback options	27
3.6.1 Floating Lookback Options	27
3.6.2 Fixed Lookback Options	29
3.6.3 The Milstein Scheme	30

3.6.4	Estimating stock price maximum	30
3.6.5	Results and Convergence Tests	33
3.7	Basket options	35
3.8	Digital options	35
3.9	Summary	36
4	American Options	37
4.1	Literature for Monte Carlo and MLMC methods for American Options	37
4.2	Introducing Mathematical Finance for American Options	38
4.3	Longstaff and Schwarz	39
4.4	The Least-Squares Approach	40
4.4.1	Explaining the method	40
4.4.2	Multilevel Monte Carlo Implementation	42
4.4.3	Level ℓ MLMC function for estimating the price of an American put option	43
4.4.4	MLMC Convergence Tests Results	45
4.4.5	Milstein and Euler-Maruyama	48
4.5	Conclusion	48
5	Acknowledgements	51
.1	Appendix: Properties of a probability measure	51
	Bibliography	51

List of Figures

2.1	Convergence Test Results for multilevel π estimator	10
2.2	Numerical Results for the π estimator	11
3.1	Simulation of 10 Brownian Motion Paths	16
3.2	Brownian bridge construction after 1, 2, 4 and 8 points have been sampled. Each step refines the previous path. [8] (Glasserman2003 p.87)	17
3.3	10 Brownian Bridge Paths and the mean of 1000 paths	18
3.4	Convergence Tests for a European Call Option with MLMC	21
3.5	European Call Option MLMC Results	22
3.6	MLMC vs MC cost for in-the-money options	25
3.7	A visualisation of the coupling for two Brownian Motion paths, a fine path in blue and a coarse path in orange.	26
3.8	Convergence Test Results for Floating Lookback Call Options	28
3.9	Result of 100 MLMC calculations for Floating Lookback Call Options	29
3.10	Fixed Lookback Euler-Maruyama Method with ad-hoc maximum	31
3.11	Convergence Test results for Fixed Lookback Options with Milstein	34
3.12	Numerical Results for a Fixed Lookback option	35
4.1	MLMC Results for the American Options	46
4.2	Convergence Test Results for Exact Simulation with an American Put Option	47

Chapter 1

Introduction

1.1 Introduction to Multilevel Monte Carlo

Monte Carlo methods are typically used when uncertainty is high-dimensional and strongly non-linear [7](Giles2018). The problem with standard Monte Carlo is that its computational cost can be very high, particularly when each sample may require approximately solving a PDE, or computations with many timesteps. Quasi-Monte Carlo (QMC) methods are one way of addressing this high cost; here samples are selected carefully to reduce the error - rather than being chosen randomly and independently. This approach is surveyed in Acta Numerica 2013 (Dick, Kuo and Sloan 2013) [13] and a brief summary is presented here.

If we wish to estimate the integral

$$I_s = \int_{[0,1]^s} f(\mathbf{x}) d\mathbf{x}$$

where s is greater than 1 and possibly large, for some integrable function f , by an n -point approximation rule of the form

$$Q_{n,s}(f) = \frac{1}{n} \sum_{i=0}^{n-1} f(\mathbf{t}_i)$$

then a rule that prescribes sample points $\mathbf{t}_0, \dots, \mathbf{t}_{n-1} \in [0,1]^s$ is called a *quasi-Monte Carlo* or QMC rule. This contrasts to the classical Monte Carlo method where the integral would be estimated with the following rule:

$$Q_{n,s}(f) = \frac{1}{n} \sum_{i=0}^{n-1} f(\mathbf{t}_i)$$

where $\mathbf{t}_0, \dots, \mathbf{t}_{n-1}$ are i.i.d (independent and identically distributed) uniform random samples from $[0, 1]^s$. The key difference is that for Quasi Monte Carlo methods, the points $\mathbf{t}_0, \dots, \mathbf{t}_{n-1}$ are chosen deterministically to be better than random, so that this leads to guaranteed error bounds and a convergence rate that may be faster than the MC rate of $O(n^{-1/2})$ for sufficiently smooth functions. [13] (Dick, Kuo and Sloan 2013 p.140)

Another way of addressing this high cost is by using Multilevel Monte Carlo methods, surveyed in Acta Numerica 2018 by M.B Giles [7] and these are the methods we will be showcasing here. The beauty of the Multilevel method is its simplicity. If we wish to estimate some random variable P , and have access to an estimator $\hat{P}_\ell$ which becomes more costly but also more accurate as ℓ , the level, increases, rather than simply using the highest and most accurate level we can achieve greater accuracy at lower cost by adopting a two-level approach where we exploit the simple identity:

$$E[P_1] = E[P_0] + E[P_1 - P_0]$$

where P_1 refers to the level 1 estimate, more costly but also more accurate, and P_0 to the level 0 estimate, less costly but also less accurate. We can then use the unbiased two-level estimator:

$$\frac{1}{N_0} \sum_{n=1}^{N_0} P_0^{(n)} + \frac{1}{N_1} \sum_{n=1}^{N_1} (P_1^{(n)} - P_0^{(n)}),$$

where $N_1 < N_0$. In the same theme, this can be generalised to the Multilevel Monte Carlo estimator: given a sequence $P_0, \dots, P_L$ which approximates P_L with increasing accuracy, but also increasing cost, we have the identity

$$E[P_L] = E[P_0] + \sum_{\ell=1}^L E[P_\ell - P_{\ell-1}]$$

for which we can use the following unbiased estimator for $E[P_L]$,

$$E[P_L] \approx \frac{1}{N_0} \sum_{n=1}^{N_0} P_0^{(0,n)} + \sum_{\ell=1}^L \left\{ \frac{1}{N_\ell} \sum_{n=1}^{N_\ell} (P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)}) \right\} \quad (1.1)$$

where the ℓ in the superscript (ℓ, n) indicates that independent samples are used at each level. If C_0, V_0 are defined as the cost and variance of one sample of P_0 , and C_ℓ, V_ℓ are the cost and variance of one sample of $P_\ell - P_{\ell-1}$, then the overall cost and variance of the multilevel estimator is $\sum_{\ell=0}^L N_\ell C_\ell$ and $\sum_{\ell=0}^L N_\ell^{-1} V_\ell$ respectively. [7] (Giles 2018, pp.3-4).

Having provided a basic introduction to the idea behind Multilevel Monte Carlo (MLMC) methods, we will now introduce the theory behind the method.

1.1.1 MLMC Theory

Geometric MLMC refers to MLMC with a geometric sequence of levels in which the cost increases exponentially with level, while both the weak error $\mathbb{E}[P_L - P]$ and the multilevel correction variance V_ℓ decrease exponentially, leading to the following theorem [7] (Giles 2018 pp. 6-7).

Theorem 1.1.1 Let P denote a random variable, and let P_ℓ denote the corresponding level ℓ numerical approximation. If there exist independent estimators Y_ℓ based on N_ℓ Monte Carlo samples, each with expected cost C_ℓ and correction variance V_ℓ and positive constants $\alpha, \beta, \gamma, c_1, c_2, c_3$ such that $\alpha \geq \frac{1}{2} \min(\beta, \gamma)$ and

$$1. \quad |\mathbb{E}[P_\ell - P]| \leq c_1 2^{-\alpha \ell}$$

$$2. \quad \mathbb{E}[Y_\ell] = \begin{cases} \mathbb{E}[P_0], & \ell = 0 \\ \mathbb{E}[P_\ell - P_{\ell-1}], & \ell > 0 \end{cases}$$

$$3. \quad V_\ell \leq c_2 2^{-\beta \ell}$$

$$4. \quad C_\ell \leq c_3 2^{\gamma \ell}$$

then there exists a positive constant c_4 such that for any $\epsilon < e^{-1}$ there are values L and N_ℓ for which the multilevel estimator

$$Y = \sum_{\ell=0}^L Y_\ell$$

has a mean-square-error with bound

$$MSE = \mathbb{E}[(Y - \mathbb{E}[P])^2] < \epsilon^2$$

with a computational complexity C with bound

$$\mathbb{E}[C] \leq \begin{cases} c_4 \epsilon^{-2}, & \beta \geq \gamma \\ c_4 \epsilon^{-2} (\log \epsilon)^2, & \beta = \gamma \\ c_4 \epsilon^{-2 - (\gamma - \beta)/\alpha}, & \beta < \gamma \end{cases}$$

This theorem will allow us to analyze some of the results from our numerical approximations.

1.1.2 MLMC Algorithm and function

In this paper we will focus solely on the geometric MLMC algorithm, and it is introduced here. Still, we note that MLMC does not require a geometric sequence of levels and the interested reader can consult section 2.6 of Giles' survey article for information on MLMC with a non-

geometric level sequence [7](Giles2018 pp.18-21). The geometric algorithm is presented below:

Algorithm 1 Geometric MLMC algorithm

```

start with  $L = 2$ , and initial target of  $N_0$  samples on levels  $\ell = 0, 1, 2, \dots$ 
while extra samples need to be evaluated do
    evaluate extra samples on each level
    compute/update estimates for  $V_\ell, \ell = 0, \dots, L$ 
    define Optimal  $N_\ell, \ell = 0, \dots, L$ 
    test for weak convergence
    if not converged; set  $L := L + 1$ , and initialise target  $N_L$ 
end while

```

Where,

$$N_\ell = \left\lceil 2\epsilon^{-2} \sqrt{V_\ell/C_\ell} \left(\sum_{\ell=0}^L \sqrt{V_\ell C_\ell} \right) \right\rceil.$$

This algorithm is not guaranteed to achieve a Mean-Squared Error (MSE) which is less than ϵ^2 as we do not have knowledge of the c_1 and c_2 constants in 1.1.1 governing the convergence of the weak error and variance as $\ell \rightarrow \infty$. They are being estimated by the algorithm above.

Chapter 2

An introduction to the MLMC functions and process through a MLMC π estimator

A classic introduction to Monte Carlo methods may involve simulating π by programming a computer to generate random pairs of Cartesian co-ordinates to represent random points in a square and then count the fraction of points which lie in a circle of radius 1. [14] (Kalos2008 p.1) This fraction should be close to $\pi/4$ and if multiplied by 4 would be a *Monte Carlo Estimate* for π . The vectorised algorithm used for this process can be seen below. 2.

Algorithm 2 Monte Carlo Estimate for π

Generate arrays of n uniform random samples:

$$X = (x_1, x_2, \dots, x_n), \quad Y = (y_1, y_2, \dots, y_n), \quad x_i, y_i \sim \text{Uniform}(0, 1)$$

Compute squared radii vectorially:

$$R = X^2 + Y^2 = (r_1, r_2, \dots, r_n)$$

Count number of points inside the circle:

$$C = \sum_{i=1}^n \mathbf{1}\{r_i \leq 1\}$$

Estimate π as:

$$\hat{\pi} = 4 \times \frac{C}{n}$$

This is in a sense a level ℓ one sample estimator for π . We generate n points, where n is a function of the level, ℓ . For example, we could have $n = 4^\ell$. So, to extend this to a Multilevel Monte Carlo estimate for π we will need to focus this concept of levels, such that the accuracy of the estimator increases with level, but also becomes more costly with level. A simple choice of levels is the number of points generated at each iteration, for instance, the level ℓ estimator could have 4^ℓ pairs of (x, y) points generated, where (x, y) are uniform as before.¹ In this context, for $N_1 > N_2$, a two level Monte Carlo estimate for π would generate N_1 coarse estimates with $4^{\ell-1}$ points and then N_2 fine estimates with 4^ℓ points, as well as N_2 coarse estimates based on subsets of the fine points of size $4^{\ell-1}$. We would then construct the coarse estimate $\hat{\pi}_c$ as an average of N_1 estimates using $4^{\ell-1}$ samples and then adjust that coarse estimate by the difference between the fine estimate $\hat{\pi}_f$ and a new coarse estimate, made with the average of N_2 coarse estimates each built using a subset of size $4^{\ell-1}$ of the points in each of the fine estimates. Here it is important to note that there are two coarse estimates, and one fine. The first is made from N_1 independently generated samples, the second built from N_2 fine estimators using N_2 subsets of size $4^{\ell-1}$ to create the coarse estimates.

The process is nicely summarised by this equation for the two-level estimator given below:

$$\hat{\pi}_{\ell=2} = \frac{1}{N_1} \sum_{n=1}^{N_1} \hat{\pi}_c - \frac{1}{N_2} \sum_{n=1}^{N_2} (\hat{\pi}_f - \hat{\pi}_c),$$

where again we note that the two $\hat{\pi}_c$ terms are determined independently. This concept can then be extended to a multilevel estimator using the unbiased MLMC estimator introduced in section 1.1. With this we will introduce the concept of a level ℓ estimator function and its use in the Multilevel Monte Carlo Method (MLMC). As stated in the theorem, we require independent estimators Y_ℓ based on N_ℓ Monte Carlo samples to utilise the MLMC algorithm. This naturally leads to the need for a level ℓ estimator function that can be used to produce these Y_ℓ terms. An example of such an M-function function for π is given on the following page 1:

¹The more accurate and more costly levels are referred to as the finer levels, and, vice versa, the less accurate and more costly are called the coarse levels.

MATLAB 1 Level ℓ estimator function for π

```

function [sums, cost] = pi_l(l, N)
r = 1; sums(1:6) = 0;
pi_f = zeros(1,N); pi_c = zeros(1, N);

for i=1:N
    if l == 0
        nf = 8^l;
        xf = 2*rand(1, nf)-1; yf = 2*rand(1, nf) - 1 % Uniform random samples
            [-1,1]
        r2f = (xf).^2 + (yf).^2; % Radius calculation for fine points
        pi_f = sum(r2f<=r^2)*4./(nf); % Pi estimate
        pi_c = 0;
    else

        nf = 8.^l; % Number of fine samples on level l
        nc = 8.^(l-1); % Number of coarse samples on level l
        xf = 2*rand(1,nf)-1; % Uniform random samples [-1, 1]
        yf = 2*rand(1,nf)-1;
        xc = xf(1:nc); % The coarse points are a subset of the fine points
        yc = yf(1:nc); % to ensure coupling.
        r2c = (xc).^2 + (yc).^2; % Compute distance to origin
        r2f = (xf).^2 + (yf).^2;
        pi_f = sum(r2f<=r^2)*4./(nf); % Pi is approximately 4 (area of square)
        pi_c = sum(r2c<=r^2)*4./(nc); % multiplied by the proportion of points in the
            circle
    end
    % Updating...
    sums(1) = sums(1) + sum(pi_f-pi_c); % E[P_l - P_{l-1}]
    sums(2) = sums(2) + sum((pi_f-pi_c).^2); % Variance
    sums(3) = sums(3) + sum((pi_f-pi_c).^3); % For Skewness Calculation
    sums(4) = sums(4) + sum((pi_f-pi_c).^4); % For Kurtosis Calculation
    sums(5) = sums(5) + sum(pi_f); % For fine estimate
    sums(6) = sums(6) + sum(pi_f.^2);
end

```

The aforementioned function [1](#) will then be used in conjunction with the M-functions, `mlmc.m`, `mlmc_test`, `mlmc_test_100`, `mlmc_plot`, and `mlmc_plot_100` all sourced from Giles' [webpage](#) and thoroughly explained in his Acta Numerica review paper [\[7\]](#)(Giles2018). A brief summary is provided here:

1. `mlmc.m`: the driver code which performs the MLMC calculation using an application-specific routine to compute $\sum_n (P_\ell^{(n)} - P_{\ell-1}^{(n)})^p$ for $p = 1, 2, 3, 4$ and a specified number of independent samples.
2. `mlmc_test.m`: a program which performs various convergence tests and checks, and then calls `mlmc.m` to perform a number of MLMC calculations.
3. `mlmc_test_n.m`: A test routine which performs n independent MLMC calculations in parallel.
4. `mlmc_plot_n.m`: This function uses the file output of the M-function `mlmc_test_n` to plot the errors for each accuracy level and plot the ratio Error/ ϵ for the RMS error, MC error, and weak error.

The weak error is computed as the absolute value of the sum of errors divided by 100. The RMS error is the root-mean-squared error, if it is less than 1 this indicates that the actual Root Mean Squared Error (RMSE) is performing better than the target RMSE, ϵ . Finally, the Monte Carlo (MC) error is simply the standard error $\sigma/\sqrt{n}$ for a Monte Carlo estimator, where σ^2 is its variance. [\[8\]](#)(Glasserman2003 p. 2)

The function that will take our level ℓ π estimator function as an input and perform MLMC using the MLMC algorithm 1 is:

`mlmc.m(mlmc_l, N0, eps, Lmin, Lmax, alpha0, beta0, varargin)`, whose inputs are defined as:

1. `mlmc_l` - the level ℓ estimator function
2. `N0` - initial number of samples
3. `eps` - desired root mean squared error
4. `Lmin` - minimum level of refinement ≥ 2
5. `Lmax` - maximum level of refinement $\geq Lmin$
6. `alpha` - the weak error is $O(2^{-\alpha l})$
7. `beta` - the variance is $O(2^{\beta l})$
8. `gamma` - the sample cost is $O(2^{\gamma l})$
9. `varargin` - any additional user inputs passed to `mlmc_l`

Upon calling the function some preliminary checks are done on the α , β , and γ values and they are estimated via linear regression if not provided or if their inputs are less than 0. For our π estimator, we will first analyse the convergence tests produced by the M-functions `mlmc_test` and `mlmc_plot`. These tests are conducted using $N = 1000$ samples across 6 levels, together with a set of target root-mean-square errors $\varepsilon = (0.005, 0.01, 0.02, 0.05, 0.1)$. The results are shown on the next page's figure 2.1.

Analysing the first row, thanks to the log-scale we can quickly note that the correction variance V_ℓ is decreasing exponentially and the numerical approximations P_ℓ are converging at an exponential rate. The second row of plots shows that the cost per sample is increasing exponentially and the kurtosis is steady between 5 and 6.5. These kurtosis values indicate that the estimator is relatively stable and the correction variance is converging. [7](Giles2018 p.23) The final row's first plot indicates that more samples are needed for more accuracy, which is fairly obvious. The second plot reveals something far more interesting though. First we note that the standard Monte Carlo cost is simply an estimate, computed as $\epsilon^{-2}V_0C_L$, where V_0 is the variance of one sample on the most coarse level, C_L the cost of $P_L - P_{L-1}$ on the most fine level, and ϵ the target root-mean squared error. [7](p.4) From our plots, it seems that the standard Monte Carlo cost scales with ϵ^2 . As greater accuracy is needed, the scaled cost increases, thus V_0C_L must be

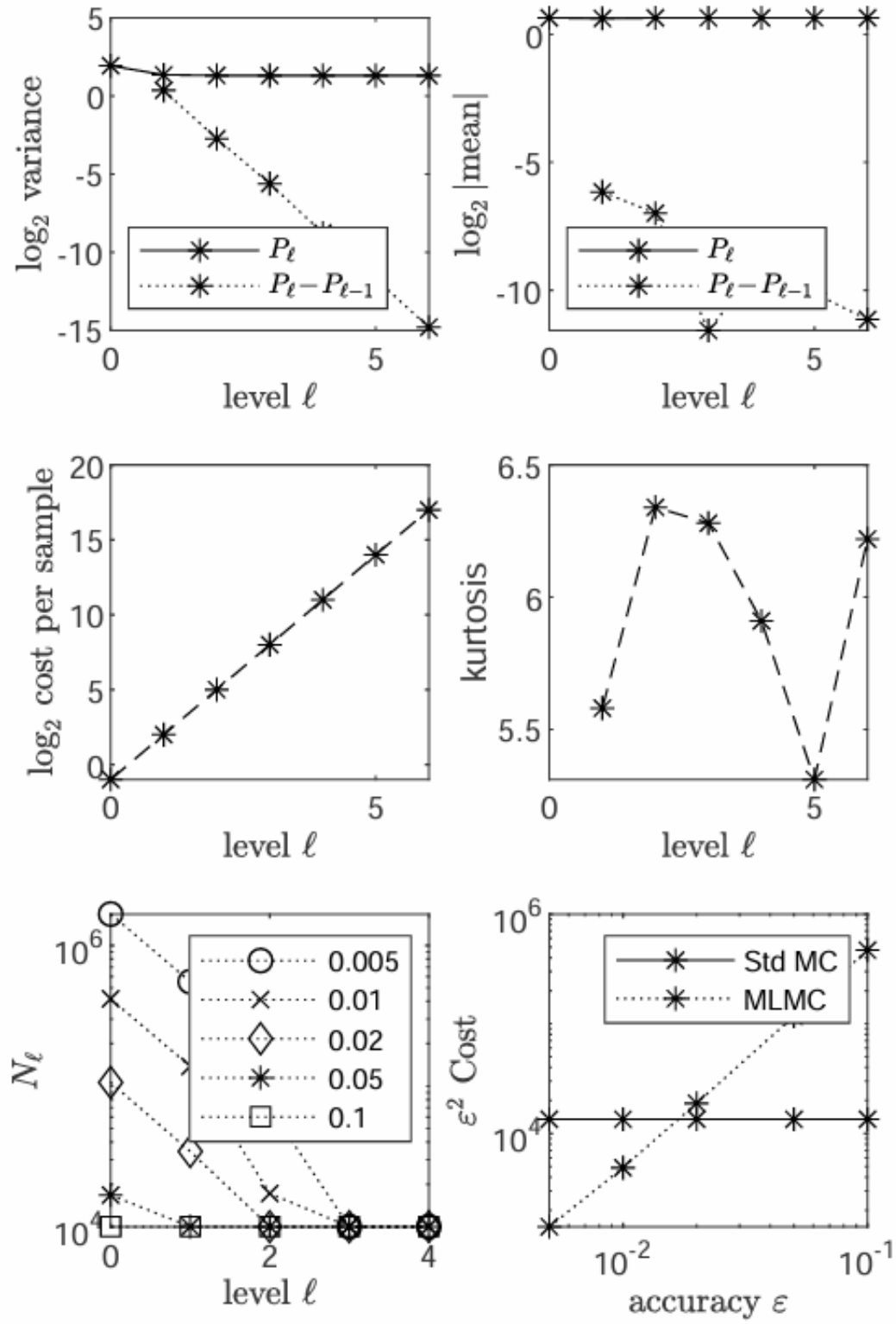
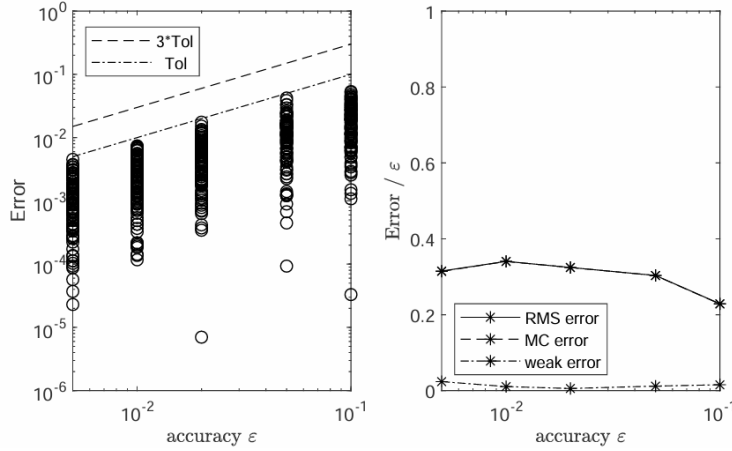


Figure 2.1: Convergence Test Results for multilevel π estimator

Figure 2.2: Numerical Results for the π estimator

some constant multiple of $1/\epsilon^2$. However, this is not the case with MLMC, for MLMC the cost seems to increase at a slower rate than ϵ^2 decreases. This results in a lower cost for MLMC when greater accuracy is required. In particular, MLMC is more efficient for $\epsilon = 0.005$ and $\epsilon = 0.01$. Still left is an analysis of the plot produced by the M-function `mlmc_plot_n` which is used with $n = 100$ for this case. In the left sided plot we see that the vast majority of errors lie below the tolerance line, indicating that the method is converging. The right hand plot plots the ratio of errors to the target root-mean-squared error, ϵ .

The MC and weak errors are near 0, indicating that the standard and weak errors are less than the desired epsilon targets. The root mean squared error is a bit larger but still below 1, indicating that it is outperforming the target RMSE.

The `mlmc_test` function also provides estimates for the α , β , and γ parameters from the MLMC theorem. In this case, the alpha value is approximately 0.9, indicating that the absolute value of the level ℓ error, $\|\mathbb{E}[P_\ell - P]\|$ is bounded, for some constant c_1 , by $c_1 2^{-0.9\ell}$. Similarly, the beta value of approximately 3 indicates that the correction variance term V_ℓ is bounded by $c_2 2^{-3\ell}$ for some positive value of c_2 . Finally, the cost is bounded by $c_3 2^{3\ell}$, for some positive c_3 .

This can be neatly summarised using Big ‘O’ notation as the absolute value of the weak error being $O(2^{-0.09\ell})$, the variance $O(2^{-3\ell})$, and the cost $O(2^{3\ell})$ where the big O refers to some positive constant multiple.

As $\beta = 3 = \gamma$ we are in the $\gamma = \beta$ situation, meaning that the computational complexity of our estimator has a bound of $O(\epsilon^{-2}(\log \epsilon)^2)$. This indicates that both the computational effort, and the contributions to the overall variance are spread approximately evenly across all levels. The $(\log \epsilon)^2$ term corresponds to the L^2 term in the expression for the total cost of an estimator if

the product $V_\ell C_\ell$ does not vary with level, which is $\epsilon^{-2} L^2 V_0 C_0 = \epsilon^2 L^2 V_L C_L$. [7] (Giles2018 p.4) **1.1.1.**

Before the end of this chapter, we will dedicate a section to the convergence test plots and how they are determined.

2.1 Convergence tests and plots explained

The M-function `mlmc_test.m` performs a set of calculations using a fixed number of samples on each level of resolution and produces six plots:

- $\log_2(V_\ell)$
- $\log_2 |\mathbb{E}[P_\ell - P_{\ell-1}]|$ and $\mathbb{E}[P_\ell]$
- $\log_2(C_\ell)$ (cost per sample)
- Kurtosis
- Samples per level N_ℓ for each target RMSE
- ϵ^2 cost comparison: MLMC (dashed) vs standard Monte Carlo (solid)

The first five plots are plotted against level ℓ and the last against target RMSE, ϵ . The first, second, and third plots relate to the α , β , and γ order of convergence terms in the MLMC theorem **1.1.1**. The fifth provides the number of samples needed for each target RMSE. However, the third and sixth require some more explanation. The third is the kurtosis estimate $\kappa = (\mathbb{E}[X^4]/\mathbb{E}[X^2])^2$, which is used as a check on the estimator, if it is too large `mlmc_test.m` will give a warning, as this is an indication that the variance estimate may be poor. [7] (Giles2018 p.23) The fifth compares the MLMC and standard Monte Carlo costs scaled by the MSE. These are calculated using the test function which calls the M-function `mlmc.m` to calculate estimates for P_ℓ from levels 0 to the max user-specified level L . At each level it computes the average of the difference between the fine and coarse estimates, and the variance of their difference. For each fine estimate, its average and kurtosis is also computed. Using these values for the weak error, correction variance and cost, the α , β , and γ exponents are estimated via linear regression.

With this information, we will now move on to computational finance applications, an area in which MLMC is better suited.

Chapter 3

Introduction to Options Pricing by Monte Carlo Methods

3.1 What are options?

Options are financial contracts giving their owner the right to buy or sell an underlying asset at a fixed price, on or before a specified future date. They are commonly referred to as derivatives, because they derive their value from their underlying assets. A stock option is a derivative whose value is dependent on the price of a stock. [11](Hull2017 p.1). A vanilla option gives its holder the right to buy or sell an underlying asset at a predetermined price (the strike price K) during a specific time period or at a specific time. A call option on a stock gives its owner the right to buy the stock at a certain price at or before a certain time. Similarly, a put option on a stock gives its owner the right to sell the stock at a certain price at or before a certain time. Options that can only be exercised at the their maturity date T , when the option expires, are called European Options. Options which can be exercised at any point up to and including the time of expiration T are called American options. In 'Futures, Options and Other Derivatives' John Hull lists the six factors that affect the price of a stock option as: [11](Hull2017 p. 234).

1. The current stock price, S_0 .
2. The strike price, K .
3. The time to expiration, T .
4. The volatility of the stock price, σ .
5. The risk free interest rate, r .

6. The dividends that are expected to be paid.

For those without a finance background, a brief overview of these terms is provided. We begin with the volatility, which, for a stock price, is a measure of how uncertain we are about future price movements. As volatility increases, the chance that the stock will either do very well or very poorly increases. The risk-free interest rate represents the expected return on an investment with no risk of financial loss. It is typically approximated by low risk assets such as US Government Bonds. As interest rates increase, the expected return required from the stock tends to increase and the present value ¹ of any future cash flow received by the holder of the option decreases. The combined impact of these two effects is to increase the value of call options and decrease the value of put options. [11](Hull2017 p. 237) A dividend is a payment made by a company to its shareholders, and they have the effect of reducing the stock price on the ex-dividend date, which is the first date after the declaration of a dividend. This reduces the value of call options and increases the value of put options. In this project we will deal solely with non-dividend paying assets. The interested reader can consult chapter 15 of John Hull [11](Hull2017 Chapter 15) for more information on the effect these factors have on option prices.

3.2 Options and their pricing

There are two fundamental option types, a *call option* gives the holder of the option the right to buy an asset by a certain date for a certain price. A *put option* gives the holder the right to sell an asset by a certain date for a certain price. The date specified in the option is known as the *expiration date* and will be denoted by T . The price specified in the contract is known as the *strike price*, this is the price at which the holder can buy or sell the asset, and will be denoted by K . Options can be either American, Bermudan or European, a European option can be exercised only at the expiration date T , whereas an American option can be exercised at any time up to the expiration date. [11](Hull2017 p. 213) Bermudan options, so named because geographically Bermuda is between America and Europe, can be exercised a finite number of times during the lifespan of the option. [1](DickmannF2015 p.1) European call options have the payoff:

$$\max(S(T) - K, 0)$$

This is because at time T , the option holder will only exercise the option if the stock price is greater than the strike price, K . In this scenario, the option holder exercises the option - buys the stock for price K - and then immediately sells it for the price S_T , pocketing the difference.

¹Present value refers to the value at time $t = 0$, i.e. due to inflation / interest the future value of the cash flow will be higher than the present value

The opposite is true for a European put option. In this case the payoff is

$$\max(K - S(T), 0),$$

because at time T , the holder will only exercise the option if the strike price K is greater than the stock price S_T . If the stock price was greater than the strike they would just sell the option for that price and it would expire worthless. The stock price process used for the analysis of these options is:

$$\frac{dS}{S} = \mu dt + \sigma dW$$

where dW is an increment of Brownian motion normally distributed with mean Δt . A more detailed explanation of Brownian motion is presented in the next section. The equation 3.2 is called geometric Brownian Motion and is the most widely used model of stock price behavior [12](Hull2021 p.301). The variable σ is the volatility of the stock price, a measure of how uncertain we are about future stock price movements. The values of both calls and puts increase as volatility increases. [12](Hull2021 p. 228) The discrete-time version of the model that we will use for our simulations is:

$$\Delta S = \mu S \Delta t + \sigma S Z \sqrt{\Delta t}$$

where $Z \sim N(0, 1)$ is a standard normal random variable. The term μ is the return, and the term multiplied by Z is the stochastic component of the return. Due to the phenomenon of risk-neutral pricing, we can take μ to be the risk free interest rate r . Risk-neutral pricing refers to the fact that the price of an option in a risk-neutral world (a world where investors do not require a premium for additional risk) is the same as the price in the real world. For more information on risk-neutral valuation please consult Section 15.7 of John Hull [12](pp.329 - 330). This stock price model comes from the assumption that the variability of the return in a short period of time, Δt , is the same regardless of the stock price. This suggests that the standard deviation of the change in a short period of time Δt should be proportional to the stock price and leads to the model outlined above. Now we will introduce the dW or $Z\sqrt{\Delta t}$ term, which is an increment of Brownian motion and encompasses all of the randomness in the price of the stock. It is quite important and a detailed explanation is given in the following section.

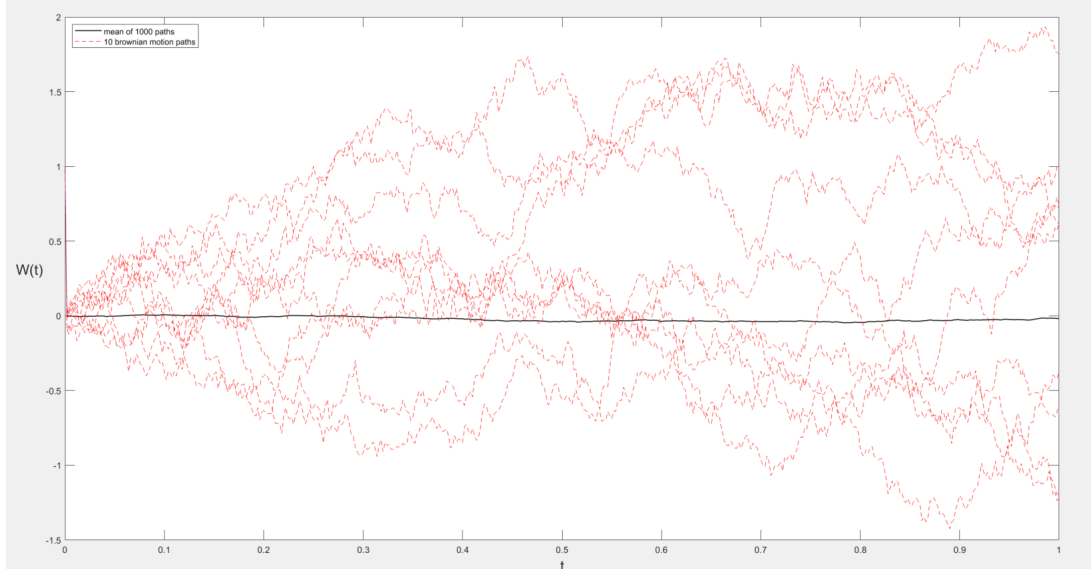


Figure 3.1: 10 Brownian Motion paths and the mean of 1000 paths

3.3 Brownian Motion

A *standard* one-dimensional Brownian motion on $[0, T]$, is a stochastic process $\{W(t) : 0 \leq t \leq T\}$ with the following properties: [8] (Glasserman2003 p.80-81)

- (i) $W(0) = 0$;
- (ii) the mapping $t \rightarrow W(t)$ is, with probability 1, a continuous function on $[0, T]$;
- (iii) the increments $\{W(t_1) - W(t_0), W(t_2) - W(t_1), \dots, W(t_k) - W(t_{k-1})\}$ are independent for any k and any $0 \leq t_0 < t_1 < \dots < t_k \leq T$;
- (iv) $W(t) - W(s) \sim N(0, t - s)$ for any $0 \leq s < t \leq T$

A consequence of (i) and (iv) is that $W(t) \sim N(0, t)$, for $0 < t \leq T$ and thus a Brownian motion for times $0 = t_0 < t_1 < \dots < t_n$ can then be constructed by setting $W(0) = 0$ and

$$W(t_{i+1}) = W(t_i) + \sqrt{t_{i+1} - t_i} Z_{i+1}, \quad i = 0, \dots, n-1$$

where the Z_i terms are independent standard normal values. For the figure above we use code from Higham's paper on the numerical simulation of SDEs [10] (Higham2001). In it the ten red lines are ten independent standard Brownian motion simulations and the black line is the average of 1000 different simulations. The average value hovers around 0 as the expected value $E[W_t]$ is equal to 0 3.1.

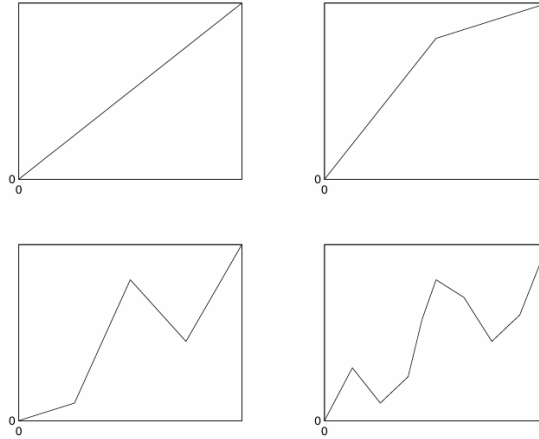


Figure 3.2: Brownian bridge construction after 1, 2, 4 and 8 points have been sampled. Each step refines the previous path. [8] (Glasserman2003 p.87)

3.3.1 Brownian Bridging

Building on this, we introduce the concept of Brownian Bridges. These are similar to Brownian motions except that with probability one they reach a specified point at a specified positive time. Brownian bridges give us greater control over the coarse structure of the simulated Brownian path and are useful for path dependent options as they add increasingly fine detail to the path at each step.

This graph from the Brownian Motion chapter in Glasserman 2003 illustrates the concept nicely, the more points that are sampled, the more fine the path becomes, though it still starts and finishes in the same place. [8] (Glasserman 2003 p.87)

To define a general Brownian Bridge, we will first define the Brownian Bridge from 0 to 0 [18] (Shreve 2004 p.176).

Definition 1 Let $W(t)$ be a Brownian motion. Fix $T > 0$. We define the Brownian bridge from 0 to 0 on $[0, T]$ to be the process

$$X(t) = W(t) - \frac{t}{T}W(T), 0 \leq t \leq T,$$

where $X(t) = X^{0 \rightarrow 0}$ is the Brownian Bridge from 0 to 0.

We can see that the Brownian motion starts at 0 with probability one as $W(0) = 0$ with the same probability and the $W(T)$ term will be multiplied by $t = 0$. It then ends at 0 almost surely² because at time T we will have $W(T) - (T/T)W(T) = 0$.

²The term almost surely refers to events with probability 1 that may not always happen.

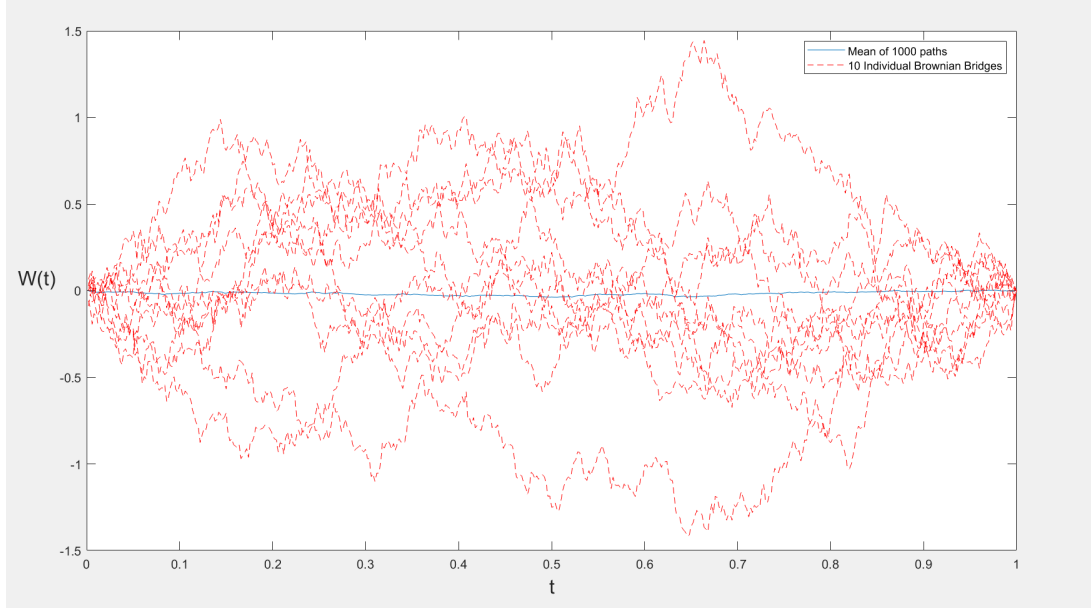


Figure 3.3: 10 Brownian Bridge Paths and the mean of 1000 paths

Building on this definition we define the Brownian Bridge from point a to point b as: [18](Shreve2004 p.176)

Definition 2 Let $X(t)$ be the Brownian Bridge from 0 to 0 in Definition 1. Fix $T > 0, a \in \mathbb{R}, b \in \mathbb{R}$. We define the Brownian Bridge from a to b on $[0, T]$ as the process

$$X^{a \rightarrow b}(t) = a + \frac{(b - a)t}{T} + X^{0 \rightarrow 0}(t)$$

.

The Brownian bridge ensures that the Brownian motion path starts at point a and ends at point b . This construction is useful for certain variance reduction techniques for path-dependent options. We will see it used later to improve the convergence of fixed Lookback options by using a Brownian Bridge interpolant to more accurately estimate the maximum of the stock price between time steps, reducing discretisation error. In the figure above 3.3, the ten red lines are ten independent Brownian Bridge simulations for a bridge starting at 0 and ending at 0. The blue line is the average of 1000 of those simulations.

3.4 Vanilla Options

Using this stock price process, introduced by [3] (Black Scholes et. al 1993), we can derive analytical solutions for the European call and put options. To do this we calculate the discounted expectation of the payoff for calls,

$$\mathbf{E}[e^{-rT}(S_T - K)^+],$$

and puts,

$$\mathbf{E}[e^{-rT}(K - S_T)^+].$$

These expectations are the discounted prices of the options, discounting refers to the fact that due to the presence of a risk free interest rate, money I have today is worth more than money I will have in the future. For this reason the expectation must be multiplied by a discount factor e^{-rT} which calculates the present value of the future price of the option. To calculate the future price of the option we use the fact that in our model $S(T)$ is log-normally distributed and the expectations are integrals with respect to the log-normal density function of $S(T)$. This expectation can then be evaluated in terms of the standard normal cumulative distribution function Φ as $BS(S(0), \sigma, T, r, K)$ with

$$BS(S, \sigma, T, r, K) = S\Phi\left(\frac{\log(S/K) + (r + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\right) - e^{-rT}K\Phi\left(\frac{\log(S/K) + (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\right),$$

which is the closed-form expectation of a European call under the Black-Scholes model [12](Hull2021 pp. 330-331). ³ This is especially useful because it allows us to determine how the Multilevel Monte Carlo method works when we know the exact solution for the price of the option.

3.4.1 Monte Carlo Simulation

First we will review the pricing with standard Monte Carlo. The solution to the Black-Scholes SDE 3.2 is: [8](Glasserman2003 p.4).

$$S(T) = S(0) \exp\left(\left[r - \frac{1}{2}\sigma^2\right]T + \sigma\sqrt{T}Z\right),$$

and we can sample from this solution to estimate the value of a call option. Given a mechanism for generating standard normal random variables Z_i , the algorithm for estimating the call price of a European option is given in [8] (Glasserman2003 p.p4-5) as:

³The price of a put can be found similarly.

Algorithm 3 Monte Carlo estimation of a European call option

```

for  $i = 1, \dots, n$  do
    Generate  $Z_i$ 
     $S_i(T) = S(0) \exp\left(\left[r - \frac{1}{2}\sigma^2\right]T + \sigma\sqrt{T} Z_i\right)$ 
     $C_i = e^{-rT}(S_i(T) - K)^+$ 
end for
 $\hat{C}_n = (C_1 + \dots + C_n)/n$ 

```

This is an unbiased estimator for any $n \geq 1$, meaning that the expectation of $\hat{C}_n$ is the call price C . It is also strongly consistent, meaning that as $n \rightarrow \infty$, $\hat{C}_n \rightarrow C$ with probability 1.

3.4.2 A Multilevel Monte Carlo Implementation

For this section we make use of Giles' `opre.m` function that has been made publicly available on his [webpage](#). Instead of using the exact solution, to demonstrate that the MLMC method's efficiency gains are present in more complicated situations, such as for models without a constant volatility, we instead use a discrete Euler approximation to the SDE 3.2. We partition the interval $[0, T]$ into m subintervals of length $\Delta t = T/m$ and over each subinterval $[t, t + \Delta t]$ simulate a transition using the discrete Euler approximation: [8](Glasserman2003 p. 7)

$$S(t + \Delta t) = S(t) + rS(t)\Delta t + \sigma S(t)\sqrt{\Delta t}Z,$$

where Z is a standard normal random variable. In this case, the levels in the MLMC method correspond to the number of timesteps used to simulate the path. The level ℓ function estimator simulates stock price paths using this method for M^ℓ timesteps. The level 0 function simulates one timestep. The amount of times these simulations are performed is determined using the MLMC algorithm detailed in Chapter 1 1.

The results of the convergence tests with $N = 1000$, $L = 5$ and target root-mean squared error (ϵ) values of 0.005, 0.01, 0.02, 0.05, and 0.1 are given on the following page 3.4. The α , β , and γ terms are estimated as 1.5, 2, and 2 respectively. As β is equal to γ we are in the region where the computational cost is bounded for some positive constant c_4 by $c_4\epsilon^{-2}(\log \epsilon)^2$. The weak error is thus $O(2^{-1.5\ell})$, the variance is $O(2^{-2\ell})$, and the sample cost is $O(2^{2\ell})$.

In the first row of the convergence plots we see this clearly; the log-scale of the plots tells us that the correction variance $V_\ell = \text{Var}(P_\ell - P_{\ell-1})$ is decreasing exponentially and the estimators P_ℓ are converging exponentially. The same is true for the cost per sample; it is increasing exponentially. The kurtosis is for the most part decreasing and does not seem to be exhibiting any strong peaks or dips, indicating that the empirical variance estimate is satisfactory [7] (Giles

2018 p. 23).

The last row shows the number of samples needed for greater accuracy, for example, at level 0, 10^8 samples are required for a RMSE of 0.005. The last plot shows the benefits of MLMC; it is 2 orders of magnitude cheaper than standard Monte Carlo. The standard Monte Carlo cost is estimated as the target Mean Squared Error multiplied by the finest level cost and the coarsest level variance [7] (Giles 2018 p. 4). We can see that MLMC is much more computationally efficient.

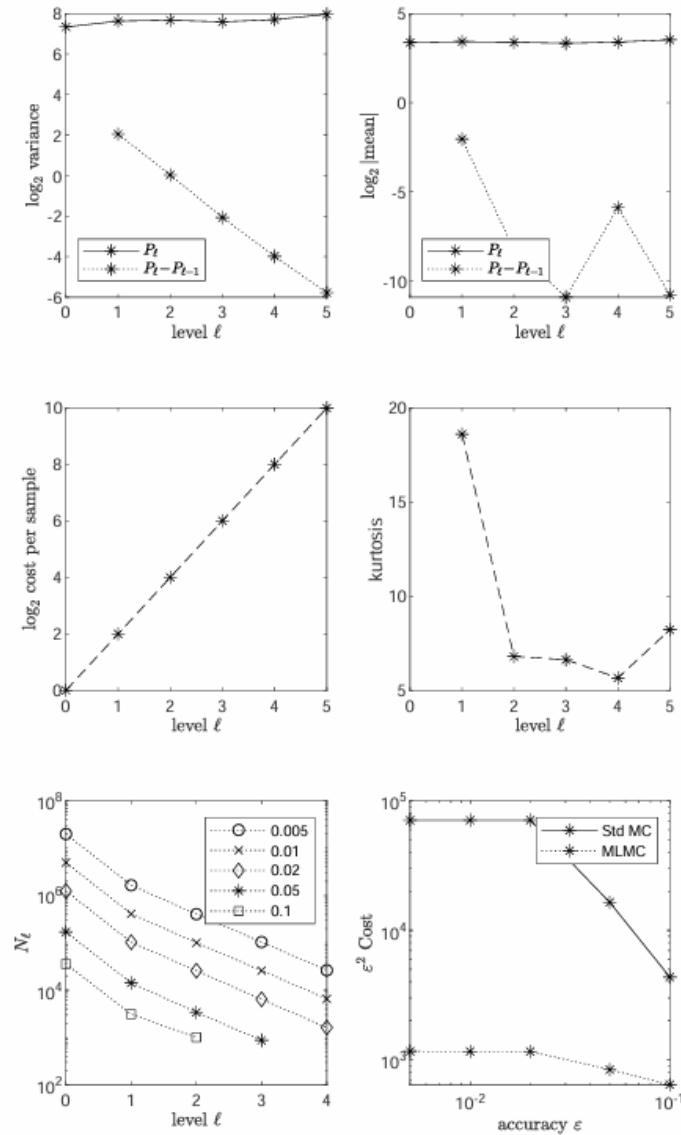


Figure 3.4: Convergence Tests for a European Call Option with MLMC

It can also be seen on the right hand plot of 3.5 that the method is very accurate, with the majority of estimates lying below the error tolerance line. As well as this, the RMSE, Weak error and Standard Errors are all below 1, indicating that they are outperforming the RMSE targets.

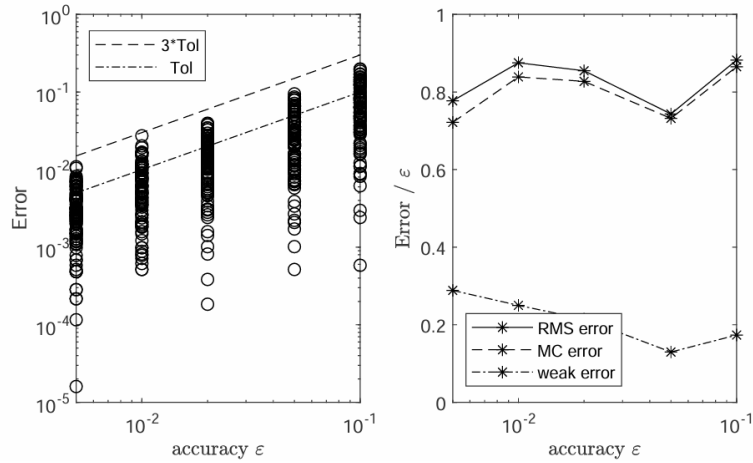


Figure 3.5: European Call Option MLMC Results

3.5 Empirical Results on a Real Life Options Dataset

To further emphasise the efficiency of the MLMC method we will demonstrate the discrepancy between standard Monte Carlo cost and MLMC costs across a variety of different option parameters using a Calls 2004 Dataset which has been sourced from the MS4028 module project.

3.5.1 Empirical Results

For this we will use a modified version of the `opre.m` file. In Giles' original paper he uses specific pre-determined default values for the variables time T , volatility σ , interest rate r , strike price K , and initial stock price S_0 , however, as we will be looping through a dataset of options which each has its own set of parameters we will need a new function that can take these parameters as an input. This new function is named `call_l` and will be described in the following section.

3.5.2 `call_l(1, N, r, sig, K, S0)`

The function first initialises the M parameter where M^ℓ corresponds to the number of timesteps on level ℓ , we use $M = 4$. [2](#)

Parameters relative to the MLMC process include:

1. The factor, M , by which the time step is refined at each level.
2. The number of time steps on the fine level ℓ , $nf = M^\ell$.
3. The number of time steps on the coarse level $\ell - 1$, $nc = M^{\ell-1}$.
4. The value by which the time step is incremented for the fine level ℓ , $hf = T/nf$.
5. The value by which the time step is incremented for the coarse level $\ell - 1$, $hc = T/nc$.

A lowercase f or c following the variable name refers to the fine and coarse level of that iteration. Using these parameters the evolution of the stock price and the expected value of the call option is simulated using the M-function `call_l` seen on the following page. [2](#).

MATLAB 2 Level ℓ function for MLMC estimation of a call option for given parameters

```

function [sums, cost] = call_l(1, N, r, sig, strikePrice, startPrice)

M = 4; T = 1./12; K = strikePrice; % T is set to 1 month to match our dataset
nf = M^1; nc = M^(1-1);
hf = T/nf;
hc = T/nc; % Time steps for each level
sums(1:6) = 0; % Initialise sums variable

for N1 = 1:10000:N
    N2 = min(10000, N-N1+1) % Ensure N > 10000
    X0 = startPrice;
    Xf = X0*ones(1, N2);
    Xc = Xf
    if 1 == 0
        dWf = sqrt(hf)*randn(1,N2);
        Xf = Xf + r*Xf*hf + sig*Xf.*dWf; % Initial stock price estimate using the
            Euler-Maruyama method
    else
        for n=1:nc
            dWc = zeros(1, N2);
            for m=1:M
                dWf = sqrt(hf)*randn(1,N2);
                dWf = dWc + dWf; % Sum the Brownian increments to ensure coupling and
                    correlation between levels
                Xf = Xf + r*Xf*hf + sig*Xf.*dWf; % Update the fine estimate
            end
            Xc = Xc + r*Xc*hc + sig*Xc.*dWc; % Update coarse estimate
        end
    end

    % Compute the option payoffs
    Pf = exp(-r*T)*max(0, Xf-K)
    Pc = exp(-r*T)*max(0, Xc-K)

    if 1 == 0
        Pc=0;
    end

    sums(1) = sums(1) + sum(Pf-Pc); % Computing E[Pf - Pc] .... etc.
    sums(2) = sums(2) + sum((Pf-Pc).^2);
    sums(3) = sums(3) + sum((Pf-Pc).^3);
    sums(4) = sums(4) + sum((Pf-Pc).^4);
    sums(5) = sums(5) + sum(Pf);
    sums(6) = sums(6) + sum(Pf.^2);
end % For loop end

cost = N*nf; % The cost is defined as number of fine timesteps by number of
    samples
end

```

The dataset we will use for this is an *in-the-money* dataset of call options. The *in-the-money* prefix means that the stock price $S(0)$ was greater than the strike price K at time 0, i.e. the beginning of the option. The M-function `mlmc_test_100.m` has been modified to output a 3 x 100 dimensional matrix with one row of 100 MLMC estimates for the payoff of the option, the second row with 100 MLMC costs, and the last with 100 estimates of the standard cost. We use this output to build a table with rows in this order:

```
|Option Parameters |AvgP |AvgC |AvgStd |P_run1 : P_run100 |C_run1 : C_run100 |Std_Run1 : Std_Run100 |
```

The result is a 308 x 108 matrix of these values, we plot the average MLMC and standard cost for each option indexed. In the Figure 3.6 we see the efficiency gains from using MLMC in practice. The average standard cost is plotted using a solid blue line. The average MLMC cost is plotted using a solid red line. On this plot, the values for MLMC costs range from 21 to 5461, and the standard MC costs range from 774 to 3,478,829. For this reason the MLMC costs appear to be almost zero in the plot. Thus we see that MLMC achieves even better cost savings in practice than in theory.

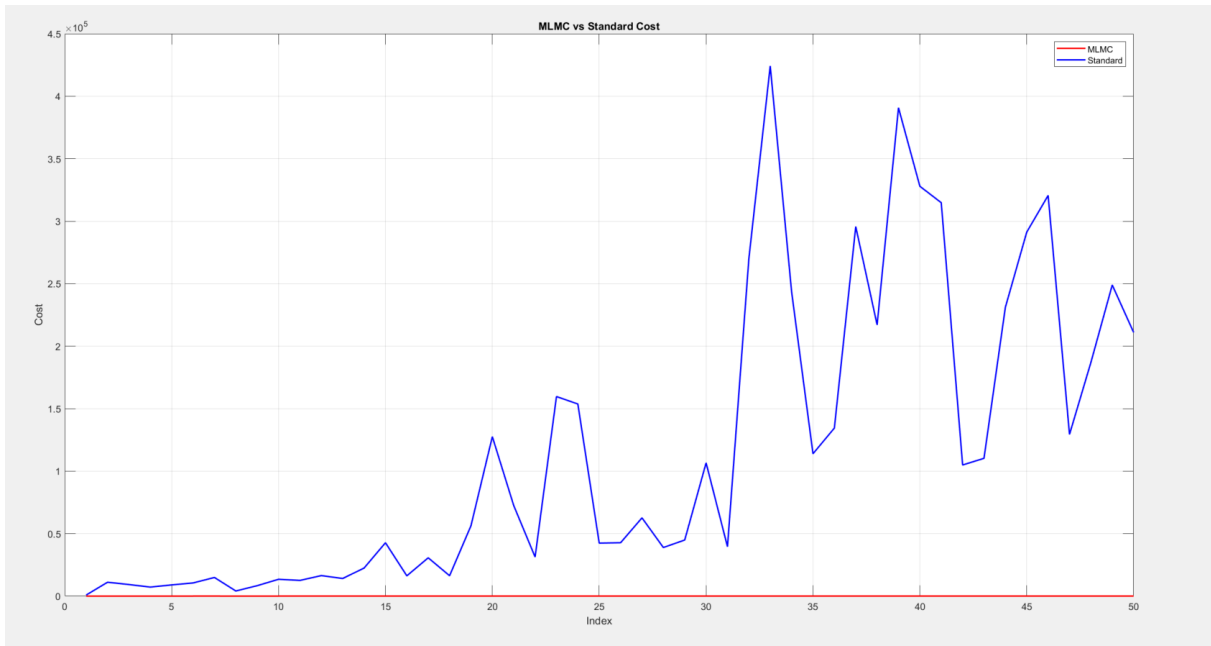


Figure 3.6: MLMC vs MC cost for in-the-money options

We end this section on vanilla call options with a discussion on the concept of coupling in MLMC.

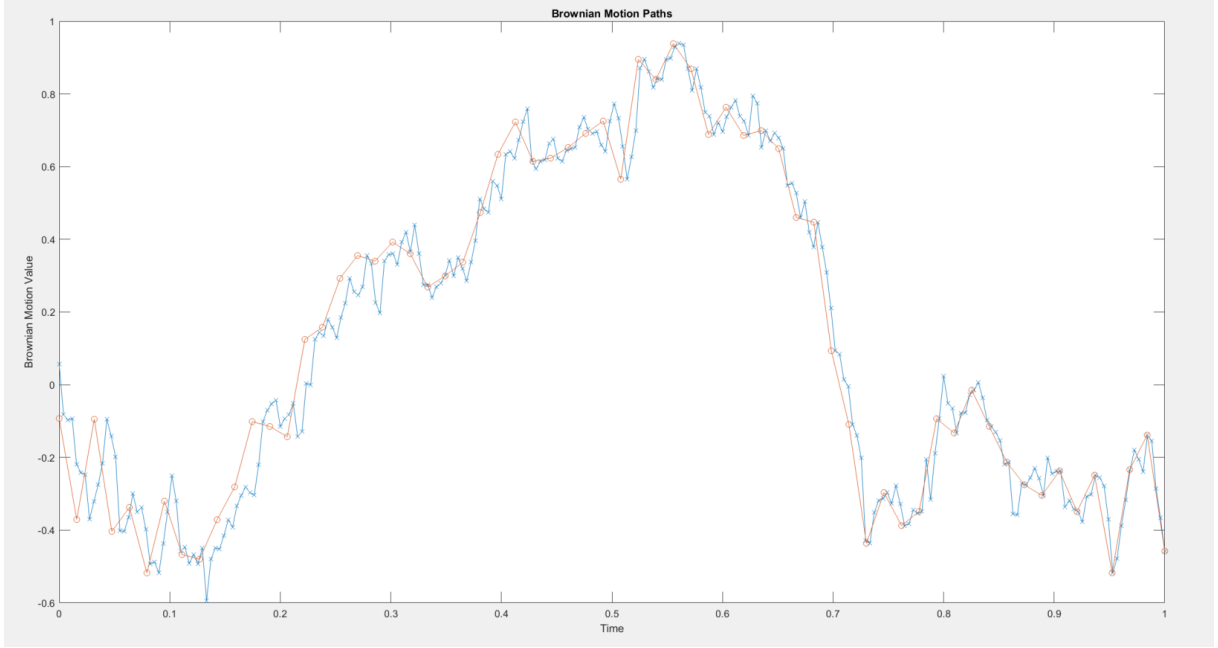


Figure 3.7: A visualisation of the coupling for two Brownian Motion paths, a fine path in blue and a coarse path in orange.

Coupling in MLMC

For a MLMC implementation something that is of vital importance is that when we compute the difference terms $P_\ell(\omega) - P_{\ell-1}(\omega)$, both estimators must correspond to the same sample ω [7](Giles 2018 p. 28). This is referred to as ‘coupling’ the estimates. In the setting with the π estimator, coupling was achieved by using a subset of the points generated for the fine estimate to compute the coarse estimate. This ensured some correlation between the two. However, achieving this same effect is somewhat more complicated when dealing with Brownian Motions and SDEs. To couple estimates in SDEs we must ensure that the coarse and fine estimates come from the same underlying Brownian motion path. [7](Giles2018 p.29) For this, we first generate a fine path and use it to obtain the coarse path by summing the factor of refinement⁴, M , Brownian motion terms. Therefore the same underlying randomness drives both paths but the fine path has a more accurate representation of this randomness. This can be visualised with the above graph of the coarse path with 64 timesteps and the fine path with 256. 3.7. The factor of refinement in this case is 4 and the plot is for the fourth level.

⁴Here the factor of refinement refers to fact that the fine path at level ℓ has M^ℓ timesteps and the coarse path has $M^{\ell-1}$ timesteps

3.6 Lookback options

There are two types of lookback options, floating and fixed. For all lookbacks the payoffs depend on the maximum or minimum asset price reached during the life of the option. We will address the two separately, first floating then fixed.

3.6.1 Floating Lookback Options

The payoffs for floating lookback call and put options respectively is:

$$S(T) - S_{\min}$$

and,

$$S_{\max} - S(T).$$

We aim to price a floating lookback call option with $S_0 = 100$, $T = 1$, $r = 0.05$, and $\sigma = 0.2$. This is done by simulating different stock price paths and numerically approximating the minimum of the stock price using the formula $\hat{S}_{\min,\ell} = (\min_n \hat{S}_n)(1 - \beta^* \sigma \sqrt{h_\ell})$. Here the term $\beta^* \approx 0.5826$ is a constant that corrects the error due to the discrete sampling of the path [5] (Giles 2008 p. 614).

The result of the convergence tests for this option, with $N = 1000$, $L = 5$, and target RMSE values of 0.01, 0.02, 0.05, 0.1 and 0.2 are given on the following page. [3.8](#)

The results are very similar to those of the European options, the correction variance, rate of convergence, and cost are approximately $O(2^{-2.27\ell})$, $O(2^{-1.76\ell})$, and $O(2^{2\ell})$ and this is verified in the plots. The MLMC cost is about two orders of magnitude cheaper than standard Monte Carlo.

The results plots are also quite similar to those for a European option, however there are higher errors in this implementation. Like the for the Europeans, in the right hand plot the majority of the estimates lie below the trend line and all of them lie below the three times tolerance line. The right hand plot displays a weak error ratio well below 1, indicating that it is lower than the target RMSE values. The RMSE and standard errors are close to the target values, as they hover around 1 in the plot.

Likewise, the results plots are extremely similar to those for a European option with the majority of the estimates lying below the error trend line and an error to RMSE ratio of approximately 1 [3.8](#).

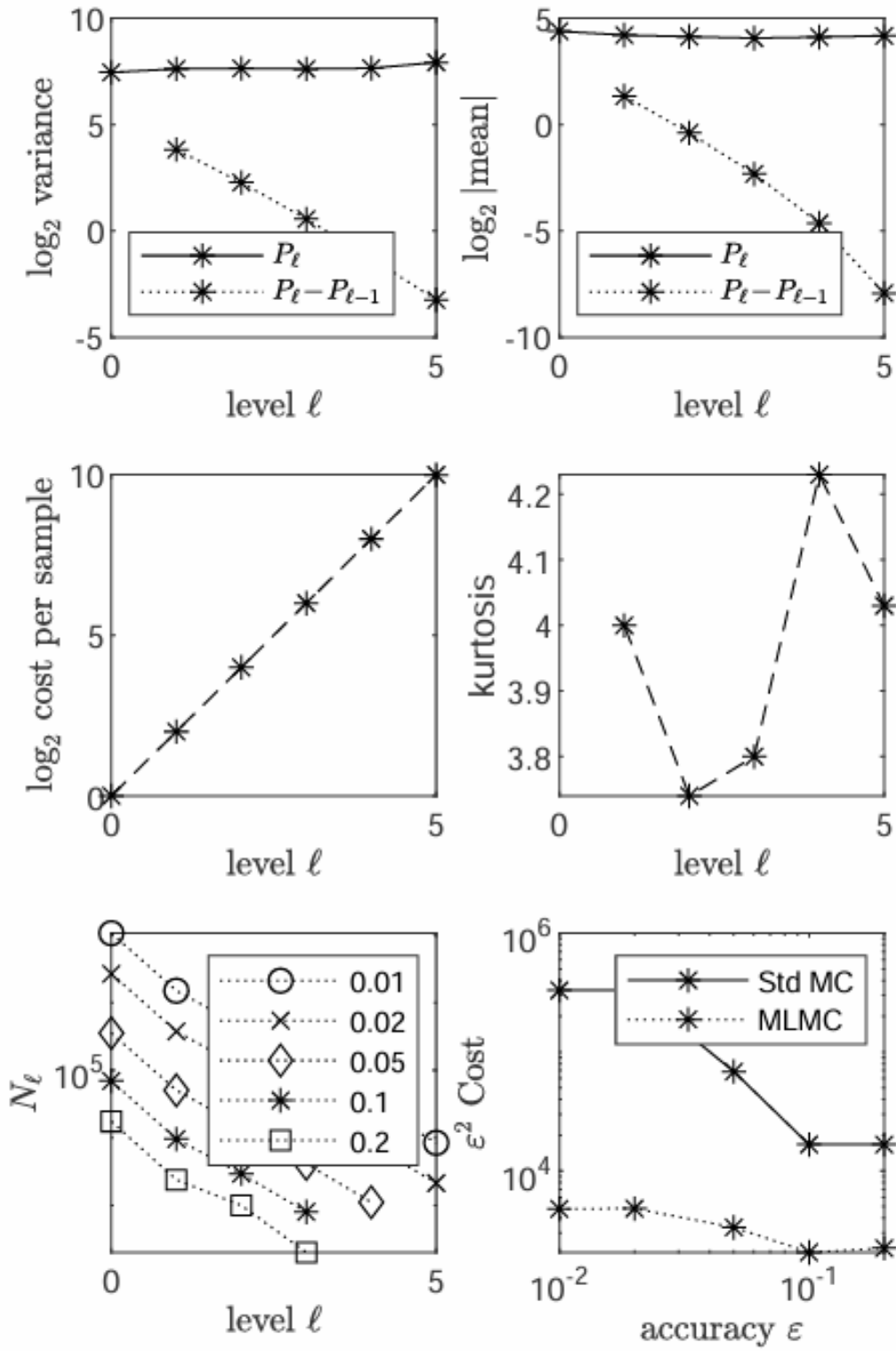


Figure 3.8: Convergence Test Results for Floating Lookback Call Options

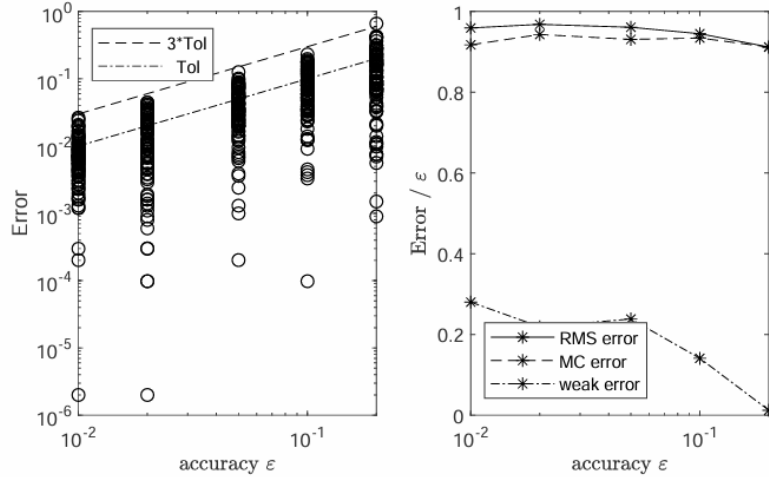


Figure 3.9: Result of 100 MLMC calculations for Floating Lookback Call Options

3.6.2 Fixed Lookback Options

Fixed lookback options are missing from Giles' M-function `opre.m` so we thought it would be a good idea to implement this ourselves. Later in the project we will address the pricing of American options and Fixed Lookbacks are useful for this purpose. Their price can be thought of as an upper bound for an American option as they allow the holder to choose the most optimal stock price for exercise over the entire time interval of the option. Finding an upper bound to an American option is known as the dual problem of American options pricing [2] (Belomestny2013 p. 2). Fixed Lookback options have respective call and put payoffs:

$$\max(S_{\max} - K, 0)$$

and,

$$\max(K - S_{\min}, 0).$$

Put call parity refers to an identity regarding European put and call options. It states that if c and p are the values of European calls and puts with strikes K on an asset S with initial price S_0 , and r is the risk-free rate, T the time of expiry, then the relationship:

$$c + Ke^{-rT} = p + S_0$$

holds. This relationship can be used to produce an analytical formula for Fixed Lookback options [12](Hull2021 pp.601-603). That formula is:

$$c_{\text{fix}} = p_{fl}^* + S_0 - K^{-rT},$$

where p_{fl}^* is the price of a put on a Floating Lookback option with the same parameters.⁵ To price Fixed Lookback options, we can simply add to the M-function `opre.m`, provided by Giles, a line of code which tracks the maximum of the stock price and a new payoff function. As in the floating case, we can use a similar ‘ad-hoc’ estimate for maximum as:

$$S_{\max, \ell}^{\wedge} = \max_n \hat{S}_n (1 + \beta \sigma \sqrt{h_\ell})$$

which results in a convergent method. In our case, we price a fixed Lookback call option with $S_0 = 100$, $T = 1$, $r = 0.05$, $\sigma = 0.2$, and $K = 100$. This gives convergence plots seen on the following page 3.10. The numerical results are that the weak error is $O(2^{-1.79\ell})$, the correction variance V_ℓ is $O(2^{-1.87\ell})$ and the cost $O(2^{2\ell})$.

3.6.3 The Milstein Scheme

A method is said to have a strong order of convergence equal to η if there exists a constant C such that

$$\mathbf{E}[|X_n - X(\tau)|] \leq C\Delta t^\eta.$$

It can be shown that the Euler Maruyama (EM) method has a strong order of convergence equal to $1/2$. The Milstein method has a strong order of convergence equal to 1 . The strong order of convergence measures the rate at which the “mean of the error” decays as $\Delta t \rightarrow 0$. In contrast, the weak error measures the “error of the means” and gives rise to the notion of weak convergence. The EM and Milstein methods have the same order of weak convergence with $\eta = 1$ [10](Higham2001 pp.532-539). However, as Milstein has a faster strong convergence this better “mean of the errors” can be used to improve the convergence of some path-dependent options. We will see how this can be done for fixed Lookbacks in the following section.

3.6.4 Estimating stock price maximum

Previously, estimating the minimum with $\hat{S}_{\min} = \min_n (\hat{S}_n - \beta \sigma \sqrt{h})$ was sufficient as this resulted in an a variance of $O(h_\ell)$ which was acceptable as in general this is the best that can be achieved with an Euler path discretisation. However, as the Milstein scheme is of a higher order, this rate of convergence can be improved [16](Kloeden1999 p. 345).

For the Black-Scholes SDE, the Milstein discretisation of the SDE is,

$$\hat{S}_{n+1} = \hat{S}_n + S r h + S \sigma \Delta W_n + \frac{1}{2} \sigma^2 S (\Delta W_n)^2$$

where σ is the volatility of the stock, r the risk free rate and ΔW_n the increment of Brownian

⁵Floating Lookback options have analytical solutions that can be derived without using the put-call parity relationship. [12] (Hull2021 p. 625)

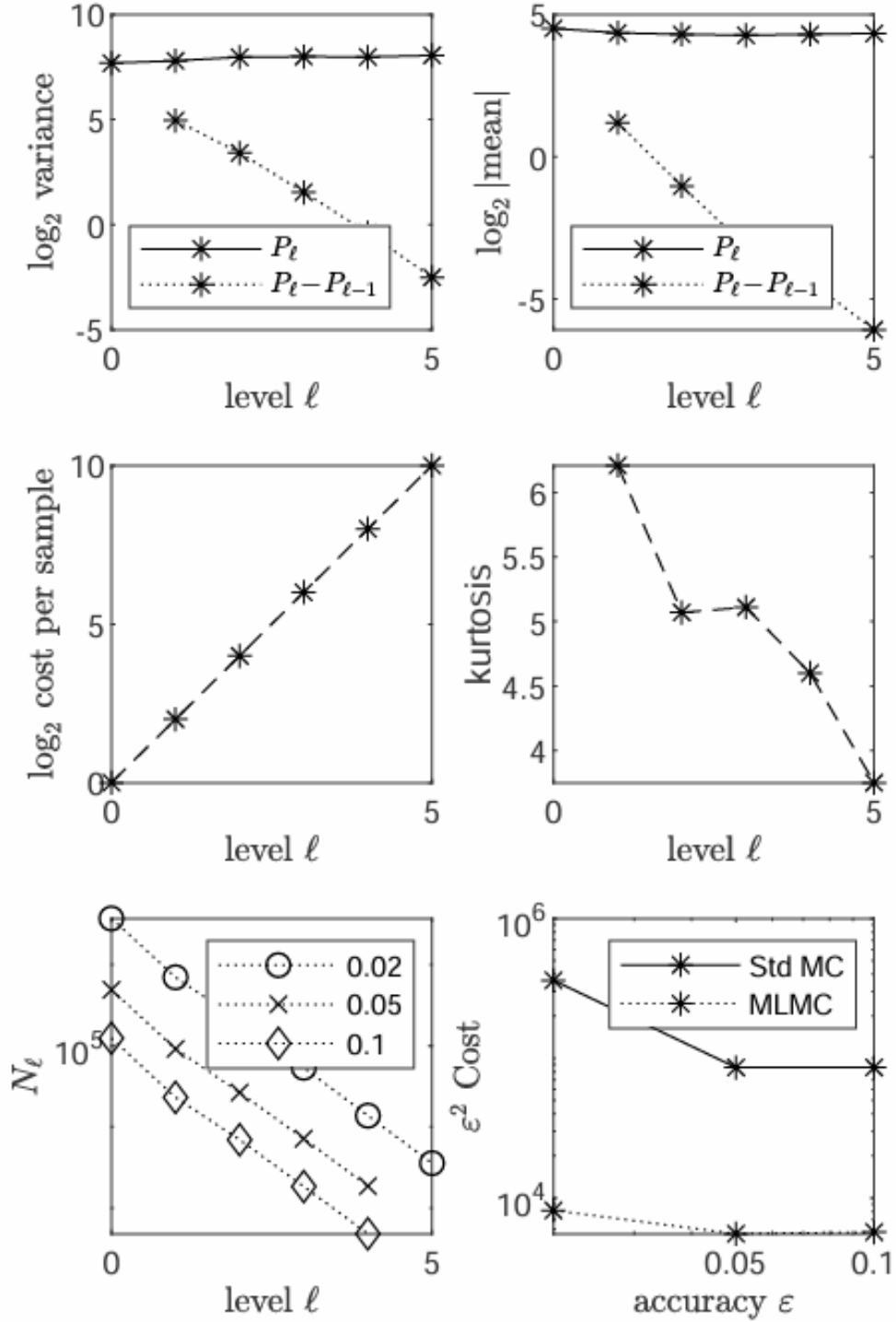


Figure 3.10: Fixed Lookback Euler-Maruyama Method with ad-hoc maximum

motion at time n . This is what we will use to simulate the stock price paths.

To avail of this improved rate of convergence we will introduce a Brownian Bridge interpolation. In section 6.4 [8](Glasserman pp366-367), Glasserman explains some standard results for sampling Brownian motions. One of those is that if we were to simply sample the maximum from a series of generated Brownian motions, for example:

$$\hat{M}^h(n) = \max \{X(0), X(h), X(2h), \dots, X(nh)\}$$

then $\hat{M}^h(n)$ is clearly the maximum of the Euler approximation to X over $[0, nh]$. The order of this approximation is $h^{1/2}$ and therefore the weak order of convergence cannot be greater than $1/2$. To avail of the higher orders of convergence available to us through Milstein approximations, we can instead generate $X(T)$ from $N(0, T)$ conditional on $X(T)$. By doing this, the process $\{X(t), 0 \leq t \leq T\}$ becomes a Brownian bridge [2](#), and we can sample the maximum of a Brownian bridge using this distribution. Given $X(T)$ the maximum is,

$$M(T) = \frac{X(T) + \sqrt{X(T)^2 - 2T \log U}}{2}$$

with $U \sim \text{Unif}[0, 1]$, independent of $X(T)$. This procedure is exact for Brownian motion, but suggests that an approximation is suitable for more general purposes. For more detail please consult section 6.4 of [8](Glasserman2003 pp.367) but in essence, given endpoints $S(i)$, and $S(i+1)$, the maximum of the interpolating Brownian bridge can then be simulated using

$$\hat{S}_{\max} = \frac{\hat{S}(i+1) + \hat{S}(i) + \sqrt{[\hat{S}(i+1) - \hat{S}(i)]^2 - 2(\hat{S}(i)\sigma)^2 h \log U_i}}{2},$$

where we assume that the stock price S is a standard Brownian Bridge between timestep i and $i+1$. The global maximum over all timesteps is then approximated from all $\hat{S}_{\max}$ values. This process is typical for Monte Carlo but not for MLMC as the difficulty lies in coupling these estimates. The standard process outline above is used for the fine path, but for the coarse path we assume a Brownian Bridge and numerically calculate the midpoint as:

$$\hat{S}_{n+1/2} = \frac{1}{2}(\hat{S}_n + \hat{S}_{n+1} - \hat{S}_n D_n)$$

where

$$D_n = W_{n+1} - 2W_{n+1/2} + W_n = (W_{n+1} - W_{n+1/2}) - (W_{n+1/2} - W_n),$$

is a $N(0, h)$ random variable which corresponds to the difference in the consecutive Brownian increments of a finer path with timestep $h/2$. Using this midpoint value, the maximum over the full timestep is the largest maxima over the two half-timesteps. This process is analogous to Giles taking the minimum over the two half-timesteps for the floating Lookback options [\[4\]](#)(Giles2006

p. 9) and results in the maximum stock price estimate of:

$$\hat{S}_{n,\max} = \max \left\{ \frac{1}{2} \left(\hat{S}_n + \hat{S}_{n+1/2} + \sqrt{(\hat{S}_{n+1/2} - \hat{S}_n) - (\hat{S}_n \sigma)^2 h \log U_{1,n}} \right), \right. \\ \left. \frac{1}{2} \left(\hat{S}_{n+1/2} + \hat{S}_{n+1} + \sqrt{(\hat{S}_{n+1} - \hat{S}_{n+1/2})^2 - (\hat{S}_n \sigma)^2 h \log(U_{2,n})} \right) \right\}$$

Where $U_{1,n}$ and $U_{2,n}$ are the uniform random variables used to compute the maximum for the two fine-path timesteps. This process ensures that the coarse and fine estimates are coupled and that their expectations are equal, as is required for MLMC. [4](Giles2006 p8-9)

3.6.5 Results and Convergence Tests

We run convergence tests with $N_0 = 2000$, minimum and maximum levels of refinement of 2 and 8 respectively. This results in the estimated values for the weak error being $O(2^{-0.82\ell})$, a correction variance of $O(2^{-1.8\ell})$, and a cost of $O(2^\ell)$. As a comparison, if we use Milstein for European options the weak error is approximately $O(2^{-0.96\ell})$, the correction variance $O(2^{-1.92\ell})$ and the cost $O(2^\ell)$. This indicates that our results are quite favourable.

Looking at the convergence test plots 3.11 the variance is decreasing and the cost increasing exponentially, which is in line with our expectations. The kurtosis is decreasing with level and is relatively stable, indicating that the empirical variance estimate is not poor. [7](Giles2018 p.23) As expected, more samples are required for lower target RMSE values. It's a bit unclear in the graph but the numeric output values for the standard and MLMC costs indicate that for epsilon values of 0.005, 0.01, 0.02, 0.05, and 0.1, MLMC is approximately 136 times cheaper for the first four and 66 times cheaper for the final value of 0.1. The results from 100 MLMC calculations for each target RMSE of $\{0.01, 0.02, 0.05, 0.1, 0.2\}$, are displayed in the graph on the page after the next 3.12. In the left hand plot we see that the method struggles for the very low RMSE value of 0.01 but for others is quite efficient, with the majority of the estimates falling below the 3x tolerance line and very few going above the 3x tolerance. The right hand plot agrees with observations from the first plot, the MC error is consistently below the target RMSE, however, the method struggles for very low RMSE targets where both the RMSE and weak error are approximately double the desired values of 0.01 and 0.02.

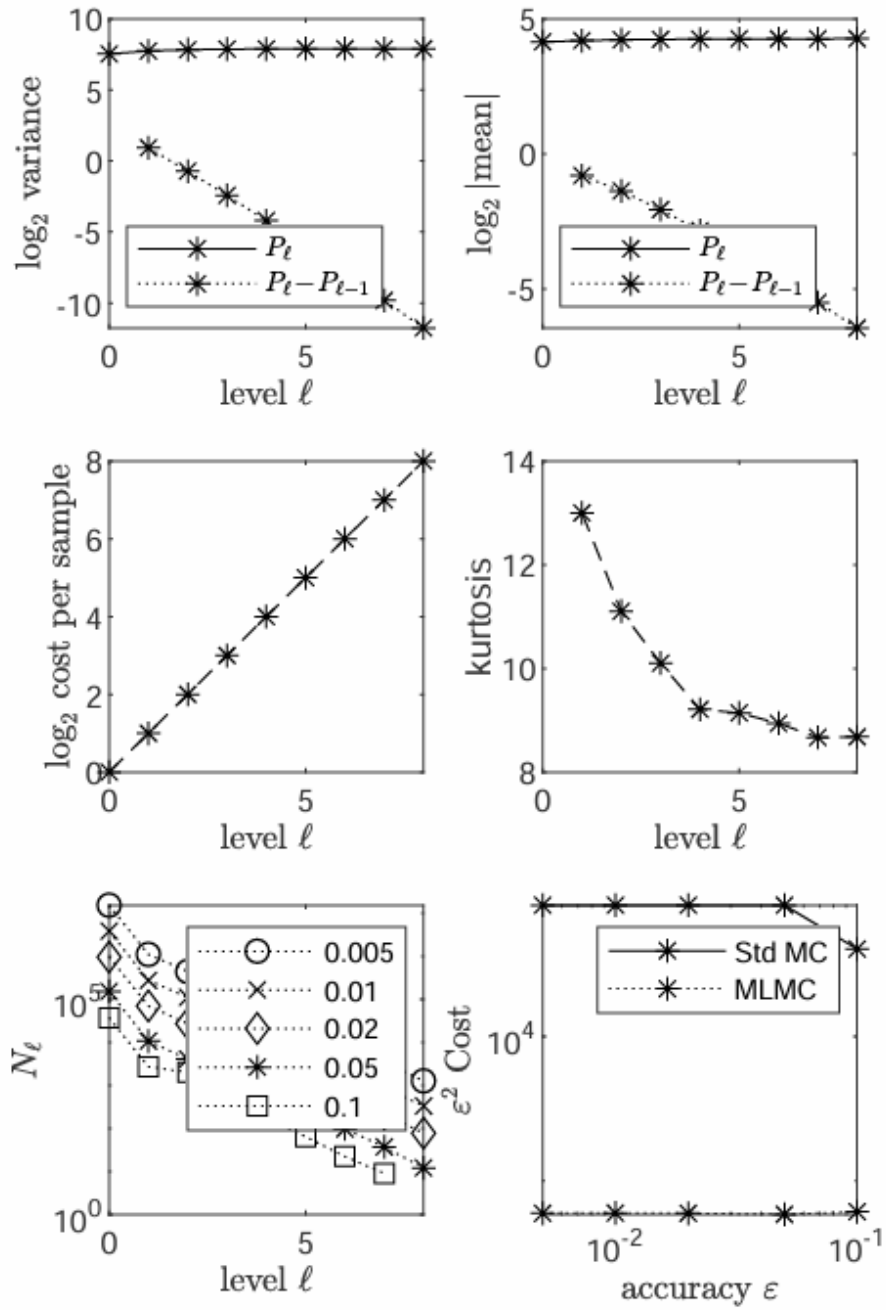


Figure 3.11: Convergence Test results for Fixed Lookback Options with Milstein

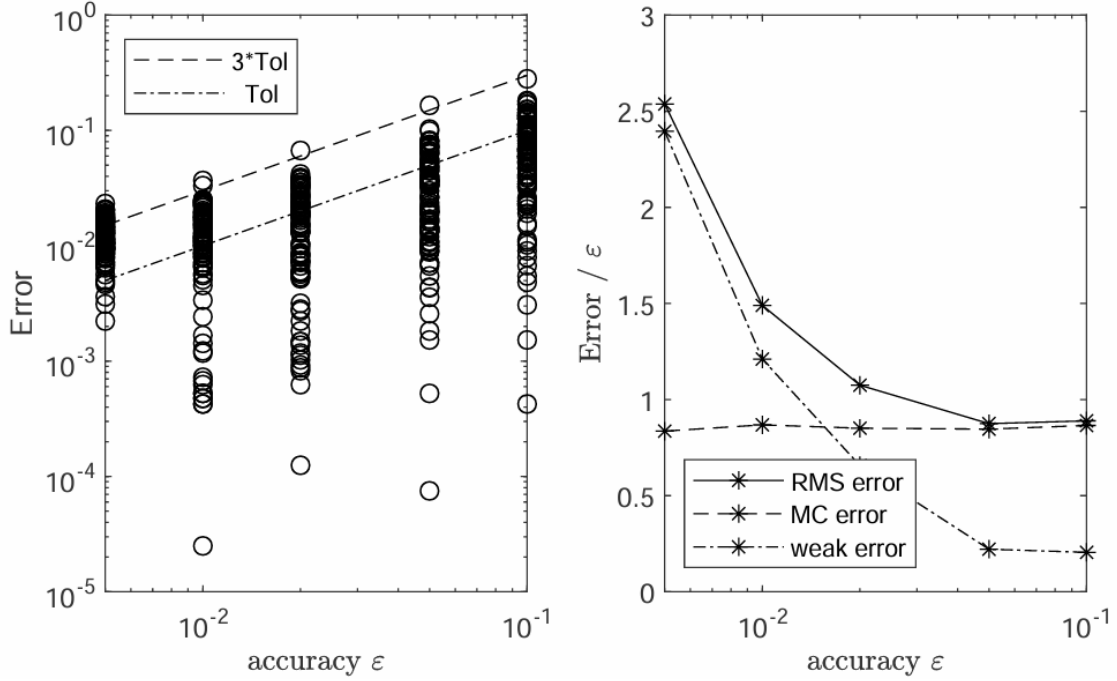


Figure 3.12: Numerical Results for a Fixed Lookback option with Milstein

3.7 Basket options

In ‘Multilevel Monte Carlo for Basket Options’ [6] (Giles 2009) Giles demonstrates that the MLMC method is easily extended to basket options based on a weighted average of several underlying quantities. We omit a further discussion here.

3.8 Digital options

Digital options have discontinuous payoff functions shown below:

$$P = K \cdot H(S(T) - K)$$

where $H(x)$ is the Heaviside function.

In this case the payoff is not continuous and thus there are problems for MLMC. The convergence tests fail to provide an order of convergence for the weak error, the alpha term is simply NaN, the variance is $O(2^{-0.9\ell})$ and the cost $O(2^\ell)$. The kurtosis is very high with a value of ≈ 363 on the finest level. In this case V_ℓ is $O(h_\ell^{1/2})$ and thus much larger sample sizes are required on the finer levels to achieve accuracy comparable to standard European options and efficiency gains

are reduced. [5](Giles 2008 p. 614).

3.9 Summary

We conclude this section with the comment that MLMC is a very cost-efficient method applicable to a variety of different option types. It is of vital importance that the fine and coarse terms are coupled to allow the MLMC estimator to iteratively pick off errors. European options are the simplest to price and show great efficiency gains when compared to standard Monte Carlo. Lookback options are more tricky but in an Euler Maruyama context benefit from transforms to the minimums and maximums. Convergence can be further improved by incorporating the Milstein scheme combined with the use of Brownian Bridging to better estimate the stock price maximum. It has also been demonstrated that MLMC can be extended to basket options based on a weighted average of several underlying stocks. Due to their discontinuous payoffs, digital options give MLMC some trouble and have very high kurtosis, indicating that the correction variance estimate may not be very good. For this reason they require much larger sample sizes to reach the same RMSE targets as other options. The final chapter will now consist of pricing American options.

Chapter 4

American Options

4.1 Literature for Monte Carlo and MLMC methods for American Options

Bermudan options are similar to American options but with the exception that the holder, rather than being able to exercise at any point during the option's lifespan can instead only exercise at specific discrete timesteps. They can be used as an approximation to American options by increasing the number of timesteps. There are three papers Giles cites on his webpage for American / Bermudan style options. These include 'Pricing Bermudan options via multilevel approximation methods' [2](Belomestny2013) where rather than considering levels corresponding to different discretization steps, they use levels related to different degrees of approximation of the continuation values. An example of this is the degrees of approximation being governed by the number of basis functions. [2] (Bolestemny et. al 2013 p.1). The methods used to produce these estimates are called primal and simulate a lower bound for the price. An alternate approach, called the dual, aims to provide an upper bound to the price. The upper bound can be represented as the Fixed Lookback options we discussed earlier, as they are akin to exercising an American option at the optimal time. There have been many approaches to reduce the complexity of this approach and in 'Multilevel dual approach for pricing American style derivatives' a multilevel version is introduced [2](Belomestny2013).

For a Vanilla American call option on stocks that pay no dividends it can be shown that the early exercise feature is worthless and the price of the call is equal to that of a European call with the same strike price [19] (Shreve1996 p. 92). As we have focused on non-dividend paying stocks throughout the report we will continue to do so in this section and attempt to price American put options. Thus we can adjust our Fixed Lookback implementation to price put

options in order to provide an upper bound to the price of an American put option. The result of this for the Fixed Lookback Milstein approach is a weak error of order $O(2^{-0.8\ell})$, a correction variance of order $O(2^{-1.8\ell})$ and a cost of order $O(2^\ell)$. It is an upper bound because unlike an American option, one can never suboptimally exercise a Fixed Lookback option and thus it will always be worth more than an American option.¹ The primal approach involves estimating the continuation value of the option at certain timesteps and exercising based off a suboptimal continuation function. As this function is suboptimal there will be times we should have exercised when we did not and therefore this approach provides a lower bound for the option price.

4.2 Introducing Mathematical Finance for American Options

Up to this point we have managed to avoid any discussion of sigma algebras and filtrations but in order to formulate the American option pricing problem we will need some understanding of these mathematical finance topics, for this reason they are introduced here.

In the American option pricing problem we will represent the stock price process, $S(t), t \in (0, T]$ as a random variable, and over time the evolving asset price $S(t) : t \in [0, T]$ is modelled as a stochastic process. [15] (Kelly2024 p. 12)

Definition 3 (Stochastic Process) *A stochastic process $S = S(t) : t \in \mathcal{T}$ is a collection of random variables parameterised over the time set $\mathcal{T} = [0, T]$, each defined on the same probability space $(\Omega, \mathcal{F}, \mathbb{P})$.*

In this definition Ω is the nonempty set, called the *sample space* which contains all possible path outcomes of the stock price. The filtration, $\mathcal{F}$ is the σ -algebra of events; sets of ω which we wish to assign a probability determined by the probability measure $\mathbb{P} : \mathcal{F} \rightarrow [0, 1]$. The properties of a probability measure can be found in the Appendix. The increasing amount of information available to a market observer regarding the evolution of the price process may be modelled via the events in $\mathcal{F}$ that, at each time $t \in [0, T]$, are decidable by the observer; known to have occurred or not occurred. For this, we equip the probability space $(\Omega, \mathcal{F}, \mathbb{P})$ with a filtration. [15](Kelly2024 p.12)

Definition 4 (Filtration) *A filtration is a family $(\mathcal{F}_t)_{t \in [0, T]}$ of increasing sub- σ -algebras of $\mathcal{F}$ such that, for all $0 \leq s \leq t \leq T$, $\mathcal{F}_s \subset \mathcal{F}_t \subset \mathcal{F}$. A stochastic process $(S(t))_{t \in [0, T]}$ is adapted to the filtration $(\mathcal{F}_t)_{t \in [0, T]}$ if, for every $t \in [0, T]$, $S(t)$ is $\mathcal{F}_t$ -measurable.²*

In words, the filtration is essentially a record of information. We can think of the σ -algebra $\mathcal{F}_1$

¹Suboptimal exercise refers to exercising the American option at a time that does not maximize its value.

² $S(t)$ being $\mathcal{F}_t$ -measurable means that every set in the sigma algebra of $S(t)$ is also in $\mathcal{F}_t$

as containing all information up to the first timestep and $\mathcal{F}_2$ as containing all the information up to the second timestep and so on and on [19](Shreve1996 p. 18). Now we are equipped with all the tools necessary to define a stopping time.

Definition 5 (Stopping Time) *Let $(\Omega, \mathcal{F}, \mathbb{P})$ be a probability space and let $\{\mathcal{F}_k\}_{k=0}^n$ be a filtration. A stopping time is a random variable $\tau : \Omega \rightarrow 0, 1, 2, \dots, n \cup \{\infty\}$ with the property that:*

$$\{\omega \in \Omega; \tau(\omega) = k\} \in \mathcal{F}_k, \forall k = 0, 1, \dots, n, \infty$$

We can now define an *American derivative security* as follows [19](Shreve1996 p.85)

Definition 6 (American Derivative Security) *An American derivative security is a sequence of non-negative random variables $\{G_k\}_{k=0}^n$ such that each G_k is $\mathcal{F}_k$ - measurable. The owner of an American derivative security can exercise at any time k , and if he does, receives the payment G_k .*

The value of the security at time k is:

$$V_k = \max_{\tau} (1+r)^k \tilde{\mathbb{E}}[(1+r)^{-\tau} G_{\tau} | \mathcal{F}_k],$$

where $\tilde{\mathbb{E}}$ refers to the risk-neutral expectation. As well as this, any stopping time τ which satisfies

$$V_0 = \tilde{\mathbb{E}}[(1+r)^{-\tau} G_{\tau}]$$

is an optimal exercise time, particularly $\tau = \min \{k; V_k = G_k\}$. [19](Shreve1996 p. 87)

The goal of the following section is to approximate this V_0 value, the value of the option at time 0.

4.3 Longstaff and Schwarz

American options are at least as valuable as their European counterparts because they can be exercised at time T and any other time within the option's lifespan. The intrinsic value of an option is the value of that option if it was exercised "now", say at time t , thus an American option can never be worth less than the payoff associated with immediate exercise [18](Shreve2004 p. 339). American calls that do not pay dividends have a worthless early exercise feature. Due to this property and because we have focused solely on options that do not pay dividends throughout the report, we will continue to do so by pricing American put options which do not have a worthless early exercise [18](Shreve2004 p. 362).

An American put with strike price K has the payoff function: ³

$$\text{Payoff} = \max(0, K - S(t)),$$

for some exercise time t .

As we have defined previously, the underlying asset will have the price process $S(t)$ given by the Black-Scholes SDE:

$$dS(t) = rS(t)dt + \sigma S(t)dW(t),$$

where the interest rate r and the volatility σ are strictly positive constants and $W(t)$ is a Brownian motion under the risk-neutral probability measure $\tilde{P}$. We will now introduce the least-squares approach in the following section.

4.4 The Least-Squares Approach

4.4.1 Explaining the method

In order to value an American-style option, we must choose between exercising and continuing at each early exercise point. The value of exercising is generally easy to determine as it's simply the intrinsic value of the option at that time. Long-Staff and Schwarz [17] provide a way of determining the value of continuing using Monte Carlo simulation.

This approach is best illustrated by a numerical example; we use the same one as in the original 2001 paper [17] (Longstaff2001 p.116-119). Let's consider a 3-year American put option on a non-dividend-paying stock that can be exercised at the end of year 1, the end of year 2, and the end of year 3. The risk-free rate is 6% per annum (continuously compounded)⁴. The current stock price S_0 is 1.0 and the strike price K is 1.1. Assume the eight paths shown in Table 4.1 are sampled for the stock price.

Path	t = 0	t = 1	t = 2	t = 3
1	1.00	1.09	1.08	1.34
2	1.00	1.16	1.26	1.54
3	1.00	1.22	1.07	1.03
4	1.00	0.93	0.97	0.92
5	1.00	1.11	1.56	1.52
6	1.00	0.76	0.77	0.90
7	1.00	0.92	0.84	1.01
8	1.00	0.88	1.22	1.34

Table 4.1: Monte Carlo simulation paths

³The option-holder has the right to sell the stock for K and thus profits if the stock is worth less than K .

⁴Continuous compounding refers to \$1 being worth $\$1e^{rT}$ at time T .

If the put option is in the money ⁵ at the 2-year point, the option holder must decide whether to exercise or wait until the next timestep. We consider only the in the money paths as this significantly improves the efficiency of the algorithm and decreases computational time [17](Longstaff2001 p.115). For these in the money paths, we assume an approximate relationship:

$$C = a + bS + cS^2,$$

where C is the value of continuing, discounted back to the 2-year point. We train a regression model on the observations on paths 1, 3, 4, 6, and 7, which are 1.08, 1.07, 0.97, 0.77, and 0.84 respectively. The regression model obtained is

$$C = -1.070 + 2.983S - 1.813S^2,$$

and the value of continuing at the 2-year point for paths 1, 3, 4, 6, and 7 is then 0.0369, 0.0461, 0.1176, 0.1520, and 0.1565 respectively.

From Table 4.1 the value of exercise at time 2 for paths 1, 3, 4, 6, and 7 is 0.02, 0.03, 0.13, 0.33, and 0.26. We combine these two calculations in the table 4.2 below and determine that for the paths which are in the money at time 2, exercise is optimal at time 2 for paths 4, 6, and 7, as the intrinsic value is greater than the continuation value.

<i>Itm Path at time 2</i>	<i>Exercise Val</i>	<i>Continuation Val</i>
1	0.02	0.0369
3	0.03	0.0461
4	0.13	0.1176
6	0.33	0.1520
7	0.26	0.1565

Table 4.2: In-the-money paths at time 2 with exercise and continuation values

Lastly, we need to determine if it is optimal to exercise at time 1, to do this we consider paths which are in the money at the 1-year point. From Table 4.1 these are paths 1, 4, 6, 7, and 8. The values of S at these points are 1.09, 0.93, 0.76, 0.92, and 0.88, respectively. The continuation values discounted back to $t = 1$ are 0 ⁶, $0.13e^{-0.06 \times 1}$, $0.33e^{-0.06 \times 1}$, $0.26e^{-0.06 \times 1}$, and 0 (as path 8 is only ITM at time 1). The least squares relationship is:

$$C = 2.038 - 3.335S + 1.356S^2,$$

which gives continuation values for paths 1, 4, 6, 7, and 8 as 0.0139, 0.1092, 0.2866, 0.1175, and 0.1533, respectively. From the original stock price matrix Table 4.1 the value of exercising at time 1 is 0.01, 0.17, 0.34, 0.18, and 0.22 respectively. These are summarised in the table below:

⁵An option is “in the money” if its intrinsic value is greater than 0.

⁶because we continued in path 1 and then ended up not exercising

<i>Itm Path at time 1</i>	<i>Exercise Val</i>	<i>Continuation Val</i>
1	0.01	0.0139
4	0.17	0.1092
6	0.34	0.2866
7	0.18	0.1175
8	0.22	0.1533

Table 4.3: In-the-money paths at time 1 with exercise and continuation values

Thus we exercise at time 1 for paths 4, 6, 7, and 8. A final table summarises the cash flows from the option using this method. It is important to note that as we cannot exercise the option more than once earlier exercise decisions overwrite previous ones.

<i>Path</i>	1	2	3
1	0.00	0.00	0.00
2	0.00	0.00	0.00
3	0.00	0.00	0.00
4	0.17	0.00	0.00
5	0.00	0.00	0.00
6	0.34	0.00	0.00
7	0.18	0.00	0.00
8	0.22	0.00	0.00

Table 4.4: Payoff values across paths and time steps

Using this table, the value of the option can be calculated by discounting each cash flow back to 0 using the risk free rate and getting the mean of the results. It is

$$\frac{1}{8}(0.07e^{-0.05 \times 3} + 10.17e^{-0.06 \times 1} + 1.18e^{-0.06 \times 1} + 0.22e^{-0.06 \times 1}) = 0.1144$$

Since this is greater than 0.10 ($K - S_0$), it is not optimal to exercise immediately.

4.4.2 Multilevel Monte Carlo Implementation

To apply MLMC to the Least Squares Monte Carlo (LSM) method we have modified a standard implementation of the Longstaff-Schwarz method, informed by online teaching materials. The multilevel components were implemented independently. We call this M-function `america_multilevel.m` and it first generates a matrix of Brownian motions of size $N \times 4^\ell$, where N is the number of paths generated and 4^ℓ the number of timesteps. Using this matrix, a second matrix is generated for the coarse level ($\ell - 1$), by summing every four Brownian increments along the timestep axis. Through this process the coarse matrix is condensed to size $N \times 4^{\ell-1}$. Next, we apply the LSM method to the coarse and fine matrices separately, generating two separate estimates of the price of the American put continuation values. These are used to generate two different estimates for the value of the option and their differences are used for the

MLMC calculation. Though it is long, we include the M-function here to assist with the readers understanding of the process.

4.4.3 Level ℓ MLMC function for estimating the price of an American put option

MATLAB 3 Part 1 of the level ℓ American Option function

```
function [sums, cost] = america_multilevel(l, N)

T = 1; r = 0.06; sig = 0.2; K = 110; S0 = 100;
M = 4;
if l == 0
    Pc = 0; nf = M^l; % Number of timesteps
    df = T/nf;
    dWf = sqrt(df)*randn(nf,N); % N : Number of paths
    Sf = cumprod([repmat(S0,1,N); exp((r-sig^2/2)*df+sig*dWf)]); % paths
    Pf = max(K-Sf(end,:),0); % put option payoff
    % going back in 2 from nf:
    for n = nf:-1:2
        Pf = exp(-r*df)*Pf; % discount
        itm_f = find(Sf(n,:) < K); % Filter for in the money paths only
        Xf = Sf(n,itm_f)'; % Set X and y with itm index for payoff
        Yf = Pf(itm_f)';
        Af = [ones(size(Xf)) (1-Xf) (1-2*Xf+0.5*Xf.^2)]; % 3 basis functions
        beta_f = Af\Yf; % linear regression
        Cf = Af*beta_f; % fine continuation value
        ind_f = itm_f((K-Xf > Cf)); % find immediate exercise
        % timesteps
        Pf(ind_f) = K-Sf(n,ind_f); % Payoff due to immediate
        % exercise
    end
else
    nf = M^l; % Number of timesteps on the fine level
    nc = M^(l-1); % Number of timesteps on the coarse level one below.
    df = T/nf; dc = T/nc; sums(1:6) = 0;
```

The function is continued on the next page:

MATLAB 4 Part 2 of the level ℓ American Option function

```

dWf = sqrt(df)*randn(nf,N); % (fine timesteps, no. paths)
dWc = reshape(dWf, M, nc, N); % How the BMs are summed to ensure coupling
dWc = sum(dWc, 1); dWc = reshape(dWc, nc, N);
Sf = cumprod([repmat(S0,1,N); exp((r-sig^2/2)*df+sig*dWf)]); % paths
Sc = cumprod([repmat(S0,1,N); exp((r-sig^2/2)*dc+sig*dWc)]); % paths
Pf = max(K-Sf(end,:), 0); % put option payoff
Pc = max(K-Sc(end,:), 0);
for n = nf:-1:2
    Pf = exp(-r*df)*Pf; % discount
    itm_f = find(Sf(n,:) < K); % Filter for in the money paths only
    Xf = Sf(n,itm_f)'; % Set X and y with itm index for payoff
    Yf = Pf(itm_f)';
    Af = [ones(size(Xf)) (1-Xf) (1-2*Xf+0.5*Xf.^2)]; % 3 basis functions
    beta_f = Af\Yf; % linear regression
    Cf = Af*beta_f; % Fine continuation value level
    ind_f = itm_f((K-Xf > Cf)); % find immediate exercise timesteps
    Pf(ind_f) = K-Sf(n,ind_f); % immediate exercise payoff
end
for n = nc:-1:2
    Pc = exp(-r*dc)*Pc; % discount
    itm_c = find(Sc(n,:) < K); % Filter for in the money paths only
    Xc = Sc(n,itm_c)'; % Set X and y with itm index for payoff
    Yc = Pc(itm_c)';
    Ac = [ones(size(Xc)) (1-Xc) (1-2*Xc+0.5*Xc.^2)]; % 3 basis functions
    beta_c = Ac\Yc; % linear regression
    Cc = Ac*beta_c; % Coarse continuation value
    ind_c = itm_c((K-Xc > Cc));
    Pc(ind_c) = K-Sc(n,ind_c); % Immediate exercise payoff
end
end
Pf = exp(-r*df)*Pf;
if l == 0
    Pc = 0;
    dc = 0;
end
Pc = exp(-r*dc)*Pc;
sums(1) = sum(Pf-Pc); sums(2) = sum((Pf-Pc).^2);
sums(3) = sum((Pf-Pc).^3); sums(4) = sum((Pf-Pc).^4);
sums(5) = sum(Pf); sums(6) = sum(Pf.^2);
cost = N*(nf+nc);
end

```

4.4.4 MLMC Convergence Tests Results

We will look at MLMC convergence in three different situations, the first where we simulate the stock price exactly, using the Black-Scholes solution, the second with the Euler-Maruyama scheme and the last using the Milstein scheme. It is important to note that what we mean by exactly is using the formula used for standard Monte Carlo simulation in Section 3.4.1 3. We do not mean that we will get an exact solution for the American option as these options do not have exact solutions. A variety of different simulation methods is used to assess the efficiency of the method for other stock price models which do not bear exact solutions such as the Heston model where the volatility is no longer constant [9](Heston 1993) or for the pricing of Asian Options based on the average stock price.

Exact Simulation

Results from the convergence tests indicate that the weak error, correction variance and cost are approximately $O(2^{-1.45\ell})$, $O(2^{-0.34\ell})$ and $O(2^{2\ell})$ respectively. The $\beta \approx 0.34$ value indicates a problem with variance decay, however there is scope for improvement as evidenced by Dickmann in his PhD thesis where he uses a clever trick to deal with this problem [1](DickmannF2015 p.50). The trick he uses is that rather than ensuring strict independence between the fine and coarse approximations he reuses the regression model from the fine level below as the model for the coarse level above. This leads to a small dependence of levels but decreases the variance. The focus of his thesis is on Bermudans, so rather than adjusting the timesteps used at each level, he adjusts each level by adding basis functions to the regression [1] (DickmannF2015 pp. 49-51). This approach of using the fine regression for the coarse timesteps a level above could be implemented here but unfortunately due to time constraints we have been unable to pursue this further.

Unlike for Dickmann's method for the Bermudan options, in our case, the regression models are retrained at each level and for this reason we calculate the cost of the MLMC estimator at each level as the number of paths N , multiplied by n_f and n_c , the number of fine and coarse timesteps respectively. Together we get that the cost is $N \times (n_f + n_c)$.

In this case, Multilevel Monte Carlo is 2x cheaper than standard Monte Carlo for RMSE targets of 0.02 and 0.01, and 1.5x cheaper for an RMSE target of 0.05. The convergence plots in Figure 4.2 indicate the problem with variance decay. in the top left plot, though it looks to decrease exponentially from level 1 to 2, afterwards the decay tapers off and does not look to decrease at an exponential rate. However, the weak error convergence is promising and looks to be converging exponentially with level. The kurtosis is increasing, likely due to the correction variance's slow convergence but this is not a huge cause for concern as we are not reaching values in the 100s as seen in the digital option case. This looks to be a symptom of the slow variance

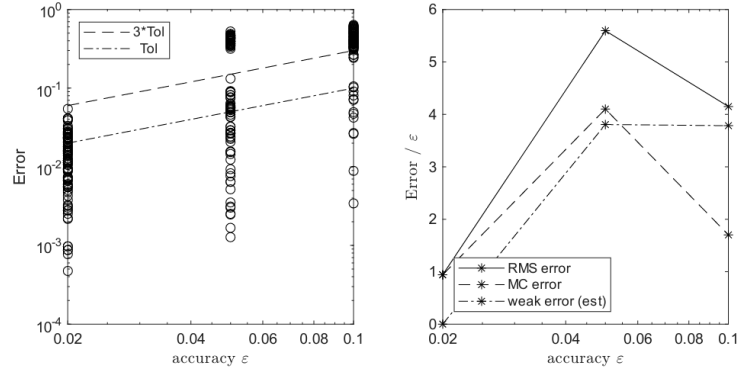


Figure 4.1: MLMC Results for the American Options

decay rather than a problem with the convergence of the estimator function itself.

Looking at the numerical results we see that for the RMSE target of 0.02, the majority of estimates fall below the trend line. Unfortunately this does not carry over for higher RMSE targets where the method seems to fail to converge as seen in Figure 4.1. In the same Figure, the second plot shows that the errors are greater than the target RMSE for all RMSE targets except for $\epsilon = 0.02$. Unfortunately, due to computational constraints, we encounter an out-of-memory error in MATLAB for RMSE targets of less than 0.2 due to the size of the matrices increasing exponentially. For this reason we have been unable to determine if the approach is successful for lower RMSE targets, this, alongside implementing Dickmann's trick, could be areas for further research.

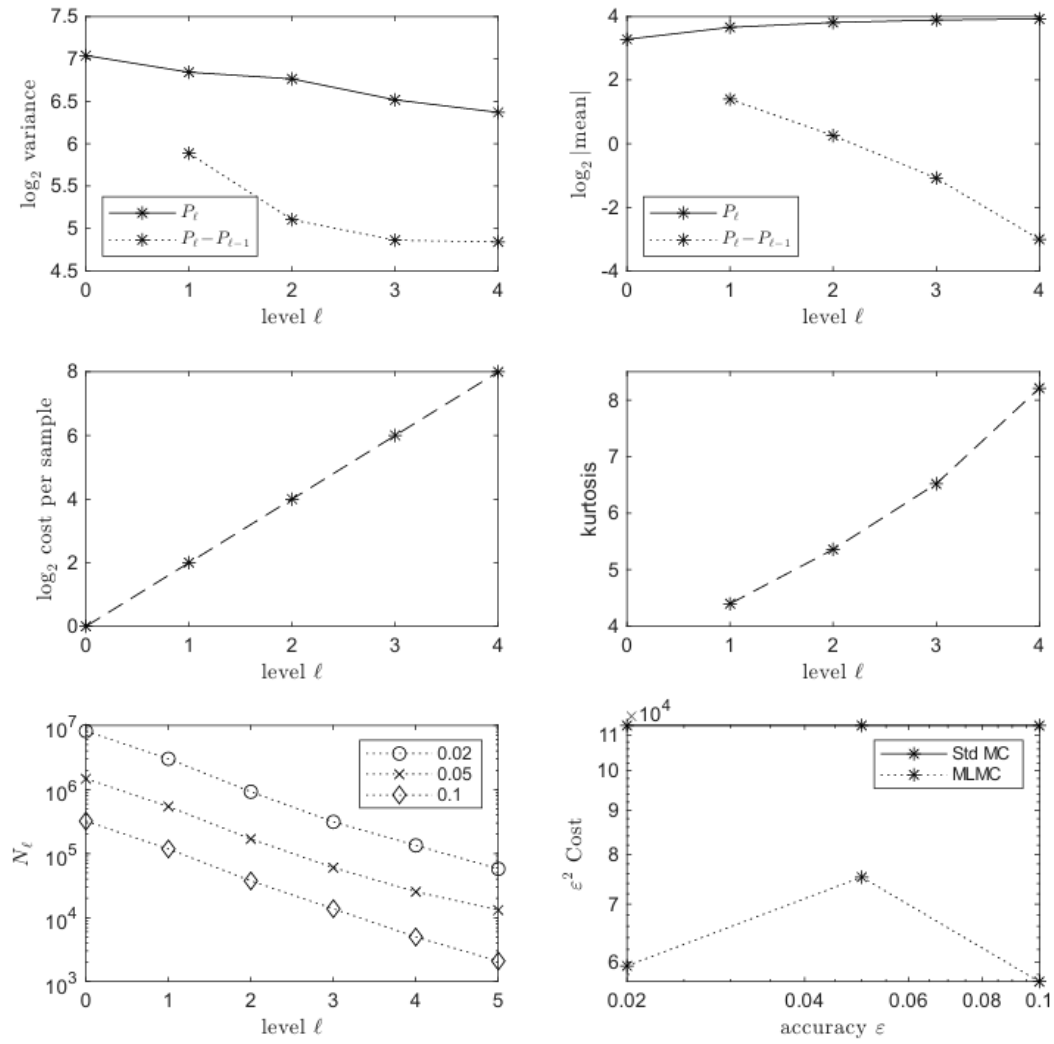


Figure 4.2: Convergence Test Results for Exact Simulation with an American Put Option

4.4.5 Milstein and Euler-Maruyama

Rather than overload this section with plots we will simply note that for the Milstein method the weak error, correction variance, and cost are approximately $O(2^{-2.45\ell})$, $O(2^{-0.3\ell})$ and $O(2^{2\ell})$ respectively. They are approximately $O(2^{-1.7\ell})$, $O(2^{-0.415\ell})$, and $O(2^{2\ell})$ for the Euler Maruyama (EM) method. With regard to cost savings compared to standard Monte Carlo the EM method is 1.84, 1.71, and 1.23 times cheaper for RMSE targets of 0.02, 0.05 and 0.1 respectively. Cost savings for the Milstein method compared to standard Monte Carlo are that Milstein is 1.94, 2.28 and 1.6 times cheaper for RMSE targets of 0.02, 0.05 and 0.1 respectively. This indicates that these savings will extend to more complicated models of the stock price.

4.5 Conclusion

On that we end the report with a summary of the chapters and results. We began by introducing the reader to the theory behind the Multilevel Monte Carlo (MLMC) method, mainly the main MLMC theorem and algorithm. We then used an example of MLMC for estimating the ratio of a circle's circumference to its diameter, π , to explain the MLMC driver functions and the convergence test results. Following this, we introduce some financial options, the log-normal Black Scholes equation used for their pricing, as well as Brownian Motion and Brownian Bridging. Next we illustrated the difference between the pricing of an option via standard Monte Carlo and MLMC using a vanilla call option. We then analysed the convergence test results and the MLMC calculation results for a standard European call option. To illustrate the immense cost-saving effect of MLMC we demonstrated the difference between pricing options using standard Monte Carlo and MLMC on a real world options dataset, we saw that comparatively the MLMC cost was almost 0. That section ended with a discussion on the importance of coupling in MLMC. Next we moved onto Lookback options and demonstrated how the Milstein scheme, alongside Brownian Bridging, could be used to price a Fixed Lookback option. We also noted that the Fixed Lookback option was similar in kind to the dual problem of American options, providing an upper bound on their price. In the final chapter we introduced the American option pricing problem and some mathematical finance terms needed to understand it. We finished the chapter with our own implementation using the exact simulation and, the Milstein and Euler-Maruyama schemes. We note that for those interested in the pricing of Fixed Lookback options, our method can be used to price them 100x cheaper than standard Monte Carlo. As well as this, for those interested in the pricing of American Put Options, our improved Fixed Lookback estimator can be used as an upper bound for their price and the improved Least Squares Monte Carlo (LSM) method can be used as a lower bound. We also note that with further research it is likely that improved cost savings can be achieved for the LSM method using the same 'trick' of reusing the fine regressions for the coarse levels for an American option as Dickmann used in his pricing of Bermudans [1] (DickmannF2015). Lastly, the code used to produce these results and the output files from running that code can both be found at this link on [GitHub](#).

Bibliography

- [1] F. D. aus Bottrop. *Multilevel Approach for Bermudan Option Pricing*. PhD thesis, Universität Duisburg-Essen, 2015.
- [2] D. Belomestny, J. Schoenmakers, and F. Dickmann. Multilevel dual approach for pricing American style derivatives. *Finance and Stochastics*, 2013.
- [3] F. Black and M. Scholes. The pricing of options and corporate liabilities. *The Journal of Political Economy*, 81, May 1973.
- [4] M. B. Giles. Improved multilevel Monte Carlo convergence using the Milstein scheme. *Monte Carlo and Quasi-Monte Carlo Methods 2006*, pages 343–358, 2006.
- [5] M. B. Giles. Multilevel Monte Carlo Path Simulation. *INFORMS*, 56:607–617, May 2008.
- [6] M. B. Giles. Multilevel Monte Carlo for Basket Options. *Winter Simulation Conference '09*, 2009.
- [7] M. B. Giles. Multilevel Monte Carlo methods. *Acta Numerica*, 2018.
- [8] P. Glasserman. *Monte Carlo Methods in Financial Engineering*. Springer, 2003.
- [9] S. L. Heston. A Closed-Form Solution for Options with Stochastic Volatility with Applications to Bond and Currency Options. *The Review of Financial Studies*, 1993.
- [10] D. J. Higham. An algorithmic introduction to numerical simulation of stochastic differential equations. *SIAM REVIEW*, 2001.
- [11] J. C. Hull. *Options, Futures, and Other Derivatives*. Pearson, 9th edition, 2017.
- [12] J. C. Hull. *Options, Futures and Other Derivatives*. Pearson Education Inc., 2021.
- [13] D. Josef, Y. F. Kuo, and I. H. Sloan. High-dimensional integration: The quasi-Monte Carlo

- way. *Acta Numerica*, 2013.
- [14] M. H. Kalos and P. A. Whitelock. *Monte Carlo Methods*. Wiley-VCH, 2008.
- [15] C. Kelly. *Computation and Simulation for Finance. An Introduction with Python*. Springer Undergraduate Texts in Mathematics and Technology, 2024.
- [16] P. E. Kloeden and E. Platen. *Numerical Solutions of Stochastic Differential Equations*. Springer, 1999.
- [17] F. A. Longstaff and E. S. Schwartz. Valuing American Options by Simulation: A Simple Least-Squares Approach. *The Review of Financial Studies*, 2001.
- [18] S. E. Shreve. *Stochastic Calculus for Finance II Continuous-Time Models*. Springer Sceince+Business Media, 2004.
- [19] S. E. Shreve. *Stochastic Calculus for Finance I, The Binomial Pricing Model*. Springer Finance, 2024.

Chapter 5

Acknowledgements

I would like to thank my supervisor, Dr. Alan Hegarty, for all the advice and help throughout this project. I've really appreciated the frequent meetings and feedback, not least helping me navigate through and understand the MATLAB MLMC code files. I would also like to extend my thanks to Prof. Mike Giles for maintaining such a detailed and helpful website of information on the MLMC method.

.1 Appendix: Properties of a probability measure

A probability measure $\mathbb{P}$ is a function mapping $\mathcal{F}$ into $[0, 1]$ with the following properties: [19](Shreve1996 p. 17)

- $\mathbb{P}(\Omega) = 1$
- If $A_1, A_2, \dots$ is a sequence of disjoint sets in $\mathcal{F}$, then

$$\mathbb{P}\left(\bigcup_{k=1}^{\infty} A_k\right) = \sum_{k=1}^{\infty} \mathbb{P}(A_k)$$